

**REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN SUPERIOR
UNIVERSIDAD NACIONAL EXPERIMENTAL
DE LAS TELECOMUNICACIONES E INFORMÁTICA**

**Informe Final: Pawcare — App Móvil para Atención Veterinaria a
Domicilio**

**Miguel Figuera C.I: 23.558.789
Iromy León C.I: V-30.243.131
Alejandra Herde C.I: V-23.711.974
Tutora: Yuly Delgado**

La Victoria, Julio de 2026

DEDICATORIA

Miguel

A Fedora y Chepino. Nos vemos en el Valhalla.

AGRADECIMIENTO

Miguel

A Rafael Hernandez y Augusto Gonzalez por todas las preguntas que respondieron sobre arquitectura para mobile, system design, local first y otro montón de inventos que metí en una aplicación solo porque sí.

A Jose Manuel da Silva; especialmente por el aprendizaje obtenido en economía y emprendimiento. Gracias a ti he logrado conseguir el capital para varios proyectos propios y universitarios también, parte de este trabajo es gracias a estas valiosas habilidades.

A algo expert, Wadie Aziz y la profesora yuly; quienes a pesar de la UNETI hacen esfuerzos sobrehumanos para mantener la calidad de la educación.

A la Uneti, por ser un mal ejemplo, gracias a esto pude canalizar mi ira y mi decepción en un proyecto como este y convertirlo en un bien para todo el ecosistema relacionado con el cuidado animal.

RESUMEN

La presente investigación tiene como objetivo el desarrollo de **Pawcare, aplicación móvil para atención veterinaria a domicilio**, segunda etapa del Proyecto Sociotecnológico iniciado con la plataforma CRM del consultorio veterinario Pawcare

(urbanización La Mora I, La Victoria, estado Aragua). El diagnóstico participativo realizado con la Dra. Génesis Conesa y su equipo evidenció que, si bien la plataforma web resolvió la gestión interna del consultorio, dos realidades limitaban su alcance: la **atención a domicilio se realiza en campo, con conectividad intermitente, y una parte importante de los usuarios no utiliza computadoras — su único dispositivo de acceso digital es el teléfono**. Como respuesta se construyó una aplicación móvil Android (Expo SDK 56, React Native, TypeScript estricto) cuyo eje arquitectónico es el enfoque **local-first**: los datos residen primero en el dispositivo (base SQLite local) y se **sincronizan con el servidor cuando existe conexión**, garantizando que la atención clínica en campo nunca dependa de la red — incluidas las alarmas de medicación y el sistema de notificaciones, que operan sin conexión. La aplicación consume el API REST del CRM (157 rutas) mediante 10 servicios de dominio y expone 74 pantallas en los dominios público, dueño y administrador. El proceso se gestionó con Scrum en sprints de dos semanas, con desarrollo guiado por pruebas (TDD) y una regla estricta de trabajo sin datos simulados: 24 suites con 108 pruebas de integración se ejecutan contra el backend real. Los resultados demuestran el cumplimiento de los objetivos planificados con una velocity estable de 27 puntos por sprint, y el impacto se valida con la comunidad mediante el Formato de Evaluación de PST provisto por la universidad, aplicado a los usuarios finales.

Palabras clave: aplicación móvil, local-first, sincronización, atención veterinaria a domicilio, React Native, Scrum, desarrollo guiado por pruebas, Investigación Acción Participativa.

INTRODUCCIÓN

En Venezuela, el teléfono inteligente es el dispositivo digital de mayor penetración: para una parte significativa de la población es, en la práctica, **el único punto de acceso a servicios digitales**, pues no posee ni utiliza computadoras personales. Cualquier solución tecnológica pensada para llegar a la comunidad — y no solo al mostrador de un negocio — debe partir de esa realidad. La primera etapa de este Proyecto Sociotecnológico entregó al consultorio veterinario Pawcare una plataforma web de gestión de relaciones con los clientes (CRM) que digitalizó sus procesos internos; la etapa que documenta este informe lleva ese sistema **al bolsillo de los usuarios**: una aplicación móvil que acompaña al equipo del consultorio en la atención a domicilio y acerca los servicios al dueño de la mascota desde su teléfono.

El desafío técnico central no fue estético sino estructural: la atención a domicilio ocurre donde la conectividad es intermitente o inexistente. Por ello, la decisión arquitectónica más importante del proyecto — registrada en el documento de decisión ADR 0001 — fue adoptar un enfoque **local-first**: la aplicación funciona primero con datos locales en el dispositivo y **sincroniza con el servidor cuando hay conexión disponible**, en lugar de exigir red permanente. Este informe final documenta las tres fases del proyecto: el diagnóstico situacional que fundamenta la necesidad (Fase I), la planificación bajo el marco Scrum (Fase II) y los resultados y logros alcanzados, incluyendo el análisis de cumplimiento de objetivos, el rendimiento del equipo y la evaluación de resultados e impacto con la comunidad (Fase III).

FASE I — DIAGNÓSTICO SITUACIONAL

Descripción del contexto

El consultorio veterinario Pawcare, dirigido por la Dra. Génesis Conesa, se encuentra ubicado en la urbanización La Mora I, La Victoria, municipio José Félix Ribas, estado Aragua. Ofrece servicios integrales de salud animal — consultas, hospedaje, peluquería canina y **atención a domicilio** — a precios accesibles, atendiendo a familias de distintos estratos socioeconómicos de la ciudad, y participa activamente en campañas de vacunación y jornadas de atención comunitaria. El equipo base lo conforman la Dra. Conesa y su asistente, apoyadas por veterinarios auxiliares en procedimientos especiales.

El diagnóstico participativo de esta etapa se realizó mediante entrevistas a profundidad con la Dra. Conesa y su asistente, observación directa de la operación — incluyendo salidas de atención a domicilio — y talleres de validación de incrementos, siguiendo el enfoque de Investigación Acción Participativa (IAP) que caracterizó todo el proyecto.

Naturaleza de la comunidad

La urbanización La Mora I es un sector residencial consolidado de La Victoria, compuesto por edificios multifamiliares y viviendas unifamiliares, con servicios públicos completos y buena conectividad vial. Alberga familias de diversos estratos socioeconómicos, muchas de ellas propietarias de mascotas. Un rasgo determinante para este proyecto: en los hogares de la comunidad **el teléfono inteligente es el dispositivo digital predominante** — la computadora personal es minoritaria — por lo que los canales digitales del consultorio deben vivir, ante todo, en el teléfono.

Antecedente: la plataforma CRM (primera etapa del PST)

La primera etapa del proyecto entregó la plataforma CRM Pawcare (Ruby on Rails 8, React, Inertia.js), que digitalizó la gestión de citas, historias clínicas electrónicas, control de vacunación, comunicación con clientes y registro de pagos del consultorio, con dos portales web (propietario y administración) y 41 requisitos funcionales documentados. Esa plataforma opera correctamente en el mostrador del consultorio; el presente proyecto parte de su API para extender el sistema hacia el campo y hacia el teléfono del usuario.

Análisis del problema

Con el CRM web en operación, el diagnóstico participativo identificó una brecha de acceso y una brecha de campo:

Causas del problema:

- La atención a domicilio — servicio distintivo del consultorio — se realiza en sectores donde la **conectividad móvil es intermitente o inexistente**, impidiendo consultar o registrar información clínica contra un sistema que exige conexión permanente.
- **Muchos usuarios del consultorio no usan computadora**: su acceso digital es exclusivamente el teléfono, por lo que un portal web pensado para escritorio les resulta ajeno o inaccesible.
- El navegador móvil no ofrece la experiencia ni las capacidades del dispositivo (almacenamiento local robusto, credenciales seguras, notificaciones), limitando la adopción del canal web en teléfonos.
- El registro de la atención en campo seguía apoyándose en notas manuales que luego debían transcribirse al sistema, duplicando trabajo y generando errores.

Efectos del problema:

- Información clínica de la atención a domicilio incompleta, tardía o perdida.
- Subutilización del CRM por parte de los dueños de mascotas sin computadora.
- Dependencia de llamadas telefónicas para agendar citas o consultar vacunas, recargando al personal del consultorio.
- Riesgo de decisiones clínicas en campo sin acceso a la historia del paciente.

Matriz de priorización de problemas

Problema	Frecuencia	Impacto	Factibilidad de solución	Prioridad
Imposibilidad de operar el sistema durante la atención a domicilio sin conexión	Alta	Crítico	Alta (arquitectura local-first)	1
Usuarios sin computadora no acceden a los servicios digitales del consultorio	Alta	Alto	Alta (app móvil nativa)	2
Registro en campo duplicado (papel → sistema)	Alta	Alto	Alta	3
Consultas telefónicas recurrentes por citas y vacunas	Media	Medio	Alta	4
Ausencia de canal móvil para adopciones, apadrinamientos y tienda	Media	Medio	Media	5

Propósito u objetivos

Objetivo General

Desarrollar una aplicación móvil **local-first** para el consultorio veterinario Pawcare que extienda la plataforma CRM hacia la atención a domicilio y hacia los teléfonos de los

dueños de mascotas, garantizando la operación sin conexión con **sincronización automática cuando exista conectividad**, mediante un enfoque de Investigación Acción Participativa.

Objetivos Específicos

1. Diagnosticar, mediante técnicas participativas, las limitaciones de acceso y de campo de la plataforma CRM existente, priorizando las necesidades de los usuarios que solo disponen de teléfono y del equipo clínico que atiende a domicilio.
2. Diseñar la arquitectura de la aplicación móvil — incluyendo la decisión local-first (ADR 0001), el modelo de datos local y la estrategia de sincronización diferida — y su sistema visual con temas claro y oscuro.
3. Desarrollar los dominios funcionales de la aplicación (público, dueño y administrador) sobre el API real del CRM, con desarrollo guiado por pruebas (TDD) y sin datos simulados en ninguna pantalla.
4. Implementar la fase inicial de la arquitectura local-first (base de datos SQLite en el dispositivo con migraciones verificadas) como fundamento de la operación sin conexión y la sincronización posterior.
5. Verificar la calidad del producto mediante pruebas de integración reales contra el backend, compuertas de calidad automatizadas y validación funcional con los informantes clave del consultorio.

Vinculación con las líneas estratégicas de desarrollo de la nación e impacto social

El proyecto se vincula con el Plan de Desarrollo Económico y Social de la Nación en su línea de transformación relativa a la ciencia y la tecnología al servicio de la solución de problemas concretos de las comunidades, y se sustenta en el siguiente marco legal:

- **Constitución de la República Bolivariana de Venezuela (1999)**: artículo 110 (interés público de la ciencia, la tecnología y la innovación), artículo 108 (incorporación del conocimiento y aplicación de las nuevas tecnologías en los centros educativos) y artículo 98 (libertad de creación y divulgación de la obra científica y tecnológica).
- **Ley Orgánica de Ciencia, Tecnología e Innovación (LOCTI, 2005)**: artículos 1 y 2 — aplicación de conocimientos académicos a la solución de problemas concretos de la sociedad y declaración de interés público de las actividades tecnológicas.
- **Ley de Infogobierno (2013)**: artículos 1 y 34 — promoción de las tecnologías de información libres y del conocimiento abierto; el proyecto publica su código fuente en repositorios públicos.
- **Ley de Ejercicio de la Medicina Veterinaria (1968) y Código Deontológico de la FCMVV**: obligaciones de registro clínico, confidencialidad y seguimiento de pacientes, que la aplicación facilita al llevar la historia clínica al punto de atención — incluso sin conexión.

El impacto social es directo: democratiza el acceso a los servicios digitales del consultorio para la mayoría que solo dispone de teléfono, y eleva la calidad del registro clínico de la atención a domicilio en sectores con conectividad limitada.

Población beneficiada

Directa:

- El equipo del consultorio Pawcare (la Dra. Génesis Conesa, su asistente y los veterinarios auxiliares), que dispone de la historia clínica y la agenda en el teléfono durante la atención a domicilio, con o sin conexión.
- Los dueños de mascotas atendidos por el consultorio — cientos de familias de La Victoria — que ahora agendan citas, consultan vacunas, historias y pagos **desde su teléfono**, sin necesidad de computadora.
- Las mascotas, beneficiarias finales de un seguimiento clínico continuo y de esquemas de vacunación al día.

Indirecta:

- Otros consultorios veterinarios de Venezuela: el código de la aplicación es público y reutilizable (municipio José Félix Ribas: más de 214.000 habitantes con demanda de servicios veterinarios similares).
- La comunidad académica de la UNETI, que recibe una referencia metodológica de desarrollo móvil local-first con calidad verificada.
- La comunidad de software libre, que obtiene una solución móvil especializada en un nicho poco atendido.

Razones del proyecto

La razón de fondo es de **efectividad del acceso**: una aplicación específica instalada en el teléfono del usuario es, en este contexto, más efectiva que cualquier portal web — funciona en el dispositivo que la gente realmente usa, opera sin conexión donde la red falla y aprovecha las capacidades del equipo (almacenamiento local, credenciales seguras). A ello se suman la razón clínica (historia del paciente disponible en el punto de atención), la razón operativa (menos llamadas, menos transcripción manual) y la

razón académica (aplicar ingeniería de software rigurosa — TDD, verificación continua, Scrum — a un problema comunitario real).

FASE II — PLANIFICACIÓN DEL PROYECTO

Factibilidad del proyecto

Factibilidad técnica. El stack elegido es maduro y de código abierto: Expo SDK 56 y React Native 0.85 con TypeScript estricto para el cliente móvil; SQLite (expo-sqlite) para la base local del enfoque local-first; el API REST existente del CRM (Ruby on Rails, 157 rutas) como backend. La documentación es extensa y las comunidades activas; el equipo domina el ecosistema React desde la primera etapa del proyecto.

Factibilidad operativa. La Dra. Conesa y su asistente participaron como informantes clave y validadoras de cada incremento; el consultorio dispone de teléfonos Android e internet para la sincronización. La aplicación reutiliza las cuentas y permisos del CRM, por lo que no crea cargas administrativas nuevas.

Factibilidad económica. El proyecto no requirió inversión adicional: herramientas de desarrollo gratuitas y de código abierto, el backend ya desplegado por la primera etapa, y equipos propios del equipo investigador. El costo marginal de operación es nulo para el consultorio.

Recursos humanos. Equipo investigador de tres estudiantes con roles Scrum definidos (ver Metodología), la tutora académica y los informantes clave del consultorio.

Recursos tecnológicos. Repositorios Git públicos (GitHub), backend Rails de referencia en ejecución, emulador Android y dispositivos reales para verificación, herramientas de calidad automatizadas (Jest, TypeScript, ESLint, react-doctor, Husky).

Metodología: Scrum

A diferencia de la primera etapa (gestionada con metodología en cascada y tablero Kanban), esta etapa adoptó **Scrum**: el producto se construyó en **sprints de dos semanas** (lunes a domingo de la semana siguiente), con planificación al inicio, reuniones de seguimiento cada 3 días, y revisión con demostración y retrospectiva al cierre de cada sprint. Los roles fueron:

Integrante	Rol
Iromy León	Scrum Master (durante todo el proyecto); gestión del proyecto por períodos, compartida con Miguel Figuera.
Alejandra Herde	QA principal: verificación de los incrementos contra la definición de terminado y validación funcional.
Miguel Figuera	Desarrollador; gestión del proyecto por períodos, compartida con Iromy León.

Justificación. El alcance de esta etapa exigía retroalimentación rápida de los informantes clave (cada dominio de la app se validaba con el consultorio) y una arquitectura que evolucionaba por fases (local-first F1–F4). Scrum, con entregas incrementales verificadas cada dos semanas y velocity medible, se ajustó a esa necesidad mejor que un ciclo secuencial. La definición de terminado incorporó las compuertas de calidad del equipo: pruebas de integración reales en verde, verificación de tipos, análisis estático sin errores, cero hallazgos de react-doctor y formato aplicado, antes de cada commit.

Cronograma de actividades

Sprint	Período	Hito principal
Sprint 1	25 may – 7 jun 2026	Concepción y paquete de diseño: requisitos (26 RF, 17 RNF), ADR 0001 local-first, diagramas, demo de interfaz.
Sprint 2	8 – 21 jun 2026	Todos los dominios funcionales

		sobre el API real con TDD; regla "nada simulado".
Sprint 3	22 jun – 5 jul 2026	Fase F1 local-first (SQLite con migraciones), selector de servidor autoalojado, calidad react-doctor; PR #1.
Sprint 4	6 – 19 jul 2026	Virtualización de listas (PR #2), sistema completo de notificaciones y alarmas de medicación local-first (PR #3), entregables académicos.

Módulos desarrollados

La aplicación expone **74 pantallas** organizadas en cuatro dominios (más dos comunes), todas consumiendo datos reales del API:

Dominio	Pantallas	Contenido
Público	14	Catálogo de productos y servicios, adopciones, apadrinamientos, checkout, contacto, términos y privacidad.
Autenticación	6	Inicio de sesión, registro, recuperación y cambio de contraseña, selector de servidor.
Dueño	26	Panel, mascotas, citas con asistente de agendamiento, historia clínica completa (consultas, vacunas, desparasitaciones, exámenes, informes), pagos, apadrinamientos, perfil, y tratamientos de medicación con alarmas.
Administrador	26	Panel con métricas, gestión de citas, consultas, vacunación, pagos, adopciones, informes médicos, recetas y exámenes de laboratorio.
Comunes	2	Centro de notificaciones (campana con badge) y preferencias de notificaciones por categoría.

Plan de acción

Actividad	Responsable	Producto verificable
Diagnóstico participativo de la brecha móvil	Equipo + informantes clave	Matriz de priorización; requisitos móviles.

Diseño de la arquitectura local-first	DEV con revisión del equipo	ADR 0001; plan de sincronización diferida.
Construcción por dominios con TDD	DEV; QA verifica	Código + 15 suites de pruebas reales.
Verificación continua	QA	Compuertas de calidad; hook pre-push; react-doctor en cero.
Validación con la comunidad	Scrum Master + equipo	Demostraciones por sprint; aplicación del Formato de Evaluación de PST de la universidad.
Gestión y seguimiento	Scrum Master	Informes parciales 1 y 2; reporte de velocity.

FASE III — RESULTADOS Y LOGROS

Resultados por fase del desarrollo

Fase de análisis de requisitos. Se documentaron 26 requisitos funcionales y 17 no funcionales específicos del cliente móvil, trazados contra las 157 rutas del API del CRM. La trazabilidad requisito → ruta → pantalla → componente alcanzó 156 de 157 endpoints (99,4 %).

Fase de diseño. Se produjo el paquete completo de diseño: ADR 0001 — Arquitectura de datos móvil (offline-first híbrido), que establece el principio rector del proyecto — **los datos viven primero en el dispositivo y se sincronizan cuando hay conexión** —; el mapa de navegación por dominios; y el sistema visual con tokens de diseño y temas claro y oscuro, validado con demo navegable.

Fase de implementación. Los cuatro dominios se construyeron con desarrollo guiado por pruebas (rojo → verde → refactorización) sobre 10 servicios de dominio contra el API real, bajo la regla estricta de **cero datos simulados**: toda pantalla visible consume el backend desde el primer día. La fase F1 del enfoque local-first quedó implementada: base de datos SQLite en el dispositivo con cuatro migraciones verificadas por pruebas. Sobre esa base se construyó el **sistema completo de notificaciones**: alarmas de

medicación con horarios anclados al «tratamiento iniciado» y tomas reajustables, centro de notificaciones in-app, recordatorios locales (citas, vacunas, desparasitación, pagos, agenda clínica), detección de cambios por sincronización y preferencias por categoría — todo operando sin conexión.

Fase de pruebas. La verificación es una propiedad del proceso, no una etapa final: 24 suites con 108 casos de integración se ejecutan contra el backend real en cada compuerta de calidad; el hook pre-push bloquea la publicación de código defectuoso; el diagnóstico react-doctor cerró en cero hallazgos; y cada incremento se validó funcionalmente en emulador Android y con los informantes clave. Como cierre del sprint 4 se ejecutó además una **revisión multi-agente de preparación para producción** (cinco revisores independientes: seguridad, corrección, experiencia de usuario, configuración de release y rendimiento) cuyos hallazgos — bloqueantes e importantes — fueron corregidos en su totalidad: protección contra registros dobles de tomas, transaccionalidad de la base local, barrera de errores global, configuración de builds (EAS), purga de datos al cerrar sesión y traducción de errores del servidor, entre otros.

Resultados cuantitativos del proyecto

Indicador	Resultado
Pantallas funcionales con datos reales	74 (4 dominios + 2 comunes)
Servicios de dominio contra el API	10
Endpoints cubiertos por la trazabilidad	156 de 157 (99,4 %)
Requisitos documentados	26 funcionales + 17 no funcionales
Suites / casos de prueba de integración real	24 / 108 — sin respuestas de API simuladas
Sistema de notificaciones y alarmas locales	Completo: centro in-app, 6 familias de recordatorios, alarmas de medicación y preferencias — operan sin conexión
Hallazgos de análisis estático y react-doctor al cierre	0
Sprints ejecutados / velocity	4 sprints; 108 puntos completados de 108

	comprometidos; promedio 27
Pull requests integrados con revisión	3 (#1, #2 y #3)
Repositorio público	github.com/TheDigitalLab-dev/pawcare-mobile

Análisis de cumplimiento de objetivos

El rendimiento del equipo se evalúa contrastando lo planificado con lo entregado, objetivo por objetivo:

Objetivo específico	Evidencia verificable	Estado
1. Diagnóstico participativo de la brecha móvil	Matriz de priorización; requisitos móviles derivados de entrevistas y observación en campo.	Cumplido
2. Diseño de la arquitectura local-first y del sistema visual	ADR 0001; plan de sincronización diferida; tokens de diseño con tema claro/oscuro; demo validada.	Cumplido
3. Desarrollo de los dominios sobre el API real con TDD y sin simulaciones	74 pantallas; 10 servicios de dominio; commits TDD auditables en el repositorio.	Cumplido
4. Implementación de la fase inicial local-first (SQLite + migraciones)	Suite de migraciones en verde; base local operativa (F1 del ADR 0001).	Cumplido (F1); F2–F4 planificadas como continuidad
5. Verificación de calidad con pruebas reales y compuertas automatizadas	24 suites / 108 pruebas contra el backend; hook pre-push; react-doctor en cero.	Cumplido

En cuanto al **rendimiento del equipo contra la planificación**: de 108 puntos comprometidos en los cuatro sprints se completaron 108 (100 %), con velocity estable (21, 34, 26 y 27) y una desviación de estimación inferior al 12 %. El sprint 4 cerró su compromiso completo con la entrega del sistema de notificaciones y las alarmas de medicación (PR #3). La única reprogramación relevante — posponer la fase F2 de sincronización para priorizar la calidad del producto — se registró como decisión explícita de alcance en la retrospectiva del sprint 3, no como desviación no controlada. El detalle sprint a sprint consta en los informes parciales 1 y 2.

Evaluación de resultados e impacto

La evaluación de resultados e impacto combina tres instrumentos:

6. **Formato de Evaluación de PST de la Comunidad:** instrumento provisto por la universidad, que el equipo presenta y aplica a los usuarios finales — la Dra. Conesa, su asistente y dueños de mascotas — para comprobar, requerimiento por requerimiento, si el software cumple las necesidades detectadas al inicio del proyecto.
7. **Indicadores objetivos del producto:** los resultados cuantitativos de la sección anterior, auditables en el repositorio público.
8. **Demostraciones validadas:** cada sprint cerró con una demostración del incremento funcionando con datos reales, con participación de los informantes clave.

El impacto esperado y observado se concentra donde el diagnóstico señaló la necesidad: el equipo clínico dispone de la información en el punto de atención a domicilio sin depender de la red, y los dueños de mascotas — incluidos quienes no usan computadora — acceden a citas, vacunas, historias y pagos desde el teléfono. La aplicación de este instrumento y la sistematización de sus resultados quedan como insumo del acta de cierre con la comunidad.

Alcance extendido del proyecto

Dado el grado de finalización alcanzado, el equipo realizó una exploración técnica **fuera del alcance del PST** — el agente de inteligencia artificial pawcare-harness (github.com/TheDigitalLab-dev/pawcare-harness) — documentada como spike en el segundo informe parcial y registrada como candidato a evolución futura del producto: un asistente conversacional dentro de la aplicación. No computó puntos en la velocity ni compromete el alcance entregado.

CONCLUSIONES

9. **El enfoque local-first fue la decisión correcta y la más importante del proyecto:** diseñar la aplicación para funcionar primero con datos locales y sincronizar cuando hay conexión responde exactamente a la realidad de la atención a domicilio en sectores con conectividad intermitente; la fase F1 (base local verificada) deja ese fundamento operativo, y las **alarmas de medicación que suenan sin conexión** son su prueba tangible: el tratamiento de la mascota no depende de la señal.
10. **El teléfono es el canal efectivo de acceso comunitario:** ante una población donde buena parte de los usuarios no utiliza computadora, una aplicación específica instalada en sus teléfonos demostró ser la vía más general y efectiva para acercar los servicios del consultorio, en lugar de esperar que la comunidad se adapte a un portal de escritorio.
11. La combinación de **TDD, pruebas de integración reales y compuertas automatizadas** convirtió la calidad en una propiedad del proceso: 108 pruebas contra el backend real y cero hallazgos de diagnóstico al cierre, verificables por cualquier evaluador en el repositorio público.
12. **Scrum con sprints de dos semanas** dio al proyecto previsibilidad medible (velocity promedio de 27 puntos, desviación de estimación < 12 %) y un mecanismo de mejora continua cuyos acuerdos — commits convencionales, pull requests, hook pre-push — quedaron implantados y auditables.
13. La Investigación Acción Participativa mantuvo el producto alineado con las necesidades reales: cada dominio se validó con los informantes clave, y el Formato

de Evaluación de la Comunidad cierra el ciclo devolviendo a los usuarios la comprobación de sus requerimientos.

RECOMENDACIONES

14. Completar las fases F2–F4 del ADR 0001 — cola de escrituras locales y **sincronización automática al recuperar conexión** — como prioridad de la continuidad del proyecto.
15. Incorporar integración continua remota (GitHub Actions) con el backend contenerizado, para re-ejecutar las compuertas de calidad en cada pull request.
16. Añadir pruebas E2E automatizadas de los flujos críticos (inicio de sesión, agendamiento, registro de pago) y medición de cobertura de código con umbral progresivo.
17. Publicar la aplicación en una tienda de distribución (o mediante APK firmado de distribución directa) y acompañar el despliegue con una jornada de capacitación a los usuarios del consultorio.
18. Aplicar periódicamente el Formato de Evaluación de PST de la Comunidad para realimentar el backlog con la experiencia real de uso.
19. Implementar las **push remotas** (avisos con la app cerrada) conforme al plan 033 documentado en el backend: registro de dispositivos y emisor Expo Push junto a los mailers existentes.
20. Evaluar, como evolución posterior al PST, la integración del asistente conversacional explorado en pawcare-harness.

BIBLIOGRAFÍA

Asamblea Nacional Constituyente. (1999). Constitución de la República Bolivariana de Venezuela. Gaceta Oficial Extraordinaria N.º 5.453.

Asamblea Nacional. (2005). Ley Orgánica de Ciencia, Tecnología e Innovación. Gaceta Oficial N.º 38.242.

Asamblea Nacional. (2013). Ley de Infogobierno. Gaceta Oficial N.º 40.274.

Beck, K. (2003). Test-driven development: By example. Addison-Wesley.

Congreso de la República. (1968). Ley de Ejercicio de la Medicina Veterinaria. Gaceta Oficial del 19 de septiembre de 1968.

International Organization for Standardization. (2011). ISO/IEC 25010: Systems and software engineering — SQuaRE — System and software quality models. ISO.

Kleppmann, M., Wiggins, A., van Hardenberg, P., y McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. Proceedings of Onward! 2019. ACM.

Schwaber, K., y Sutherland, J. (2020). La Guía de Scrum: la guía definitiva de Scrum. Scrum.org.

ANEXOS

Anexo A. Requisitos funcionales y no funcionales del cliente móvil

Anexo B. Demo de interfaz de usuario (Demo UI)

Anexo C. Diagramas de arquitectura

Anexo D. Rutas del API Rails consumidas por la aplicación (rails-routes.txt)

ANEXO A

Requisitos funcionales y no funcionales del cliente móvil

Fuente: docs/mobile-requirements.md — 26 RF y 17 RNF trazados a las 157 rutas del API

PawCare Mobile — Requisitos funcionales y no funcionales

Basado en [mobile-routes.md](./mobile-routes.md) · 157 rutas API · App movil **Android** (Expo / React Native)

Alcance de plataforma: solo **Android**. Sin soporte para iOS.

Convenciones

Tipo	Formato
RF- (funcionales)	Descripcion · Prioridad · Justificacion · Criterios de aceptacion
RNF- (no funcionales)	`RNF-{Categoria}-{NNN}: Titulo` + enunciado directo (ver Parte 2)

Prioridades RF: **Alta** (MVP / legal) · **Media** (operacion completa) · **Baja** (mejora o alcance secundario)

Parte 1 — Requisitos funcionales

RF-PUB-01 — Informacion de servicios clinicos

Descripcion

La app debe mostrar el catalogo de servicios veterinarios ofrecidos por la clinica, consumiendo la informacion publica del backend, sin requerir autenticacion.

Prioridad

Media

Justificacion

Permite al usuario descubrir la oferta antes de registrarse o agendar una cita. Rutas: `GET /services`, `GET /pages/services`.

Criterios de aceptacion

- El usuario puede abrir la pantalla de servicios sin iniciar sesion.
- Se listan los servicios retornados por `GET /services` con nombre, descripcion y precio cuando el API los provea.
- Ante error de red o respuesta vacia se muestra un estado vacio o mensaje de error comprensible.
- Existe un acceso claro hacia registro, login o agendamiento de cita desde servicios.

RF-PUB-02 — Tienda de productos y checkout

Descripcion

La app debe permitir consultar productos, ver detalle, crear un pedido y subir comprobante de pago, como flujo publico de compra.

Prioridad

Media

Justificacion

Habilita venta de insumos desde el movil. Rutas: `GET /pages/products`, `GET /public/products`, `GET /public/products/:id`, `GET /pages/checkout`, `POST /public/product\_orders`, `POST /public/product\_orders/:id/upload\_proof`.

Criterios de aceptacion

- El usuario puede listar productos via `GET /public/products`.
- El usuario puede ver detalle de un producto via `GET /public/products/:id`.
- El checkout muestra resumen del pedido antes de confirmar.
- Al confirmar, la app envia `POST /public/product\_orders` con los datos requeridos por el API.
- Tras crear el pedido, el usuario puede adjuntar comprobante con `POST /public/product\_orders/:id/upload\_proof` (imagen o archivo segun soporte del backend).
- Se muestran mensajes de exito o error segun la respuesta del servidor.
- El stock o indisponibilidad del producto se refleja en UI cuando el API lo indique.

RF-PUB-03 — Adopcion de mascotas

Descripcion

La app debe exponer el programa de adopcion: landing informativo, listado de mascotas disponibles y ficha de detalle.

Prioridad

Media

Justificacion

Canal publico para promover adopciones responsables. Rutas: `GET /adoption`, `GET /adoption/pets`, `GET /adoption/pets/:id`.

Criterios de aceptacion

- La landing de adopcion (`GET /adoption`) muestra informacion general y enlace al listado.
- El listado (`GET /adoption/pets`) muestra mascotas disponibles con datos basicos (nombre, especie, edad, estado).

- El detalle (`GET /adoption/pets/:id`) muestra ficha completa, requisitos y accion de contacto o solicitud segun defina el API.
- Solo se muestran mascotas retornadas por el backend; no hay datos hardcoded de produccion.

RF-PUB-04 — Patrocinios (acceso publico)

Descripcion

La app debe permitir consultar campanas de patrocinio, ver detalle, gastos asociados y crear o actualizar un patrocinio desde el dominio publico.

Prioridad

Baja

Justificacion

Canal de donaciones y transparencia de gastos. Rutas: `GET /sponsorships`, `GET /sponsorships/:id`, `GET /sponsorships/:id/expenses/:expense\_id`, `POST /sponsorships`, `PATCH /sponsorships/:id`, `PUT /sponsorships/:id`.

Criterios de aceptacion

- El usuario puede listar patrocinios activos via `GET /sponsorships`.
- El detalle muestra meta, progreso e historial cuando el API lo incluya.
- El usuario puede abrir un gasto especifico via `GET /sponsorships/:id/expenses/:expense\_id`.
- El formulario de alta envia `POST /sponsorships` con validacion de campos obligatorios.
- La edicion envia `PATCH` o `PUT` a `/sponsorships/:id` y refleja cambios en pantalla.

RF-PUB-05 — Contacto y contenido legal

Descripcion

La app debe mostrar paginas de contacto, terminos de uso y politica de privacidad como contenido informativo publico.

Prioridad

Alta

Justificacion

Requisito legal y de confianza para usuarios de la app. Rutas: `GET /pages/contact`, `GET /pages/terms`, `GET /pages/privacy`.

Criterios de aceptacion

- Las tres pantallas son accesibles sin autenticacion.

- El contenido se obtiene del backend o se renderiza segun respuesta de las rutas `pages#\*`.
- Contacto muestra medios oficiales de la clinica (telefono, email, direccion) cuando el API los provea.
- Terminos y privacidad son legibles en movil (scroll, tipografia adecuada).
- Enlaces a terminos y privacidad estan disponibles desde registro y checkout.

RF-OWN-01 — Autenticacion y gestion de sesion

Descripcion

La app debe permitir login, logout, consulta del usuario autenticado y renovacion silenciosa de sesion mediante tokens.

Prioridad

Alta

Justificacion

Base de todo el dominio Owner y Admin. Rutas: `GET /auth`, `POST /auth/login`, `DELETE /auth/logout`, `GET /auth/current\_user`, `POST /auth/refresh`.

Criterios de aceptacion

- La pantalla de bienvenida (`GET /auth`) ofrece acceso a login y registro.
- Login envia credenciales a `POST /auth/login` y persiste el token de forma segura en el dispositivo.
- Tras login exitoso, la app navega al dashboard o pantalla principal del rol correspondiente.
- `GET /auth/current\_user` se usa al iniciar la app para restaurar sesion valida.
- `POST /auth/refresh` se invoca antes de expiracion o ante 401 recuperable, sin interrumpir al usuario innecesariamente.
- Logout (`DELETE /auth/logout`) invalida sesion local y remota, y redirige a pantalla publica o auth.
- Credenciales invalidas muestran error claro sin revelar si el email existe.

RF-OWN-02 — Registro de dueno y recuperacion de contrasena

Descripcion

La app debe permitir registrar un dueno de mascota y completar el flujo de olvido y restablecimiento de contrasena.

Prioridad

Alta

Justificacion

Onboarding de nuevos usuarios y recuperacion de acceso. Rutas: `POST /auth/register\_owner`, `POST /auth/forgot\_password`, `GET /auth/reset\_password/:token`, `POST /auth/reset\_password`.

Criterios de aceptacion

- El formulario de registro envia `POST /auth/register\_owner` con validacion client-side de email, password y campos obligatorios.
- Tras registro exitoso, el usuario puede iniciar sesion o queda autenticado segun respuesta del API.
- "Olvide mi contrasena" envia email via `POST /auth/forgot\_password` y confirma la accion en UI.
- El deep link o ruta con token abre pantalla de nueva contrasena (`GET /auth/reset\_password/:token`).
- Restablecer contrasena envia `POST /auth/reset\_password` y maneja token invalido o expirado.
- Passwords cumplen reglas minimas indicadas por el backend.

RF-OWN-03 — Perfil del dueño

Descripcion

El dueño autenticado debe poder ver, editar su perfil, cambiar contrasena y eliminar su cuenta.

Prioridad

Alta

Justificacion

Autogestion de datos personales y cumplimiento de derechos del usuario. Rutas: `GET /profile`, `PATCH /profile`, `PATCH /profile/password`, `DELETE /profile`.

Criterios de aceptacion

- `GET /profile` muestra datos actuales del usuario autenticado.
- Edicion envia `PATCH /profile` y actualiza la UI con la respuesta del servidor.
- Cambio de contrasena usa `PATCH /profile/password` pidiendo contrasena actual y nueva.
- Eliminar cuenta requiere confirmacion explicita antes de `DELETE /profile`.
- Tras eliminar cuenta, la sesion se cierra y no quedan tokens locales.

RF-OWN-04 — Dashboard del dueño

Descripcion

La app debe presentar un resumen del estado del dueño: citas proximas, pagos, alertas medicas y accesos rapidos.

Prioridad

Alta

Justificacion

Punto de entrada principal post-login. Rutas: `GET /dashboard`, `GET /dashboard-summary`.

Criterios de aceptacion

- El dashboard consume `GET /dashboard` y/o `GET /dashboard-summary`.
- Muestra al menos: proxima cita, cantidad de mascotas y pagos pendientes cuando el API los incluya.
- Los accesos rapidos navegan a mascotas, citas, historial medico y pagos.
- El dashboard solo es accesible con sesion de dueno valida.
- Ante fallo del API se muestra estado de error con opcion de reintentar.

RF-OWN-05 — Gestion de mascotas (dueno)**Descripcion**

El dueno debe poder listar, crear, editar, eliminar sus mascotas y gestionar la foto de perfil de cada una.

Prioridad

Alta

Justificacion

Entidad central del ecosistema PawCare. Rutas: `GET /my-pets`, `GET /pets`, `GET /pets/:id`, `GET /pets/new`, `GET /pets/:id/edit`, `POST /pets`, `PATCH /pets/:id`, `PUT /pets/:id`, `DELETE /pets/:id`, `POST /pets/:id/photo`, `DELETE /pets/:id/photo`.

Criterios de aceptacion

- Listado disponible via `GET /my-pets` o `GET /pets` con datos consistentes.
- Creacion usa `POST /pets` con campos obligatorios validados.
- Edicion usa `PATCH` o `PUT` en `/pets/:id`.
- Eliminacion usa `DELETE /pets/:id` con dialogo de confirmacion.
- Subida de foto usa `POST /pets/:id/photo` desde camara o galeria.
- Eliminar foto usa `DELETE /pets/:id/photo`.
- El dueno solo ve y modifica sus propias mascotas (errores 403/404 manejados en UI).
- Desde el detalle se accede a historial medico y agendamiento de citas.

RF-OWN-06 — Citas veterinarias (dueno)**Descripcion**

El dueño debe poder listar citas, ver detalle, agendar, editar, confirmar y cancelar citas usando disponibilidad de veterinarios y días.

Prioridad

Alta

Justificacion

Funcionalidad core de la clinica movil. Rutas: `GET /my-appointments`, `GET /owner-appointments-list`, `GET /owner-appointments/:id`, `POST /owner-appointments`, `PATCH /owner-appointments/:id`, `POST /owner-appointments/:id/confirm`, `POST /owner-appointments/:id/cancel`, `GET /owner-available-vets`, `GET /owner-available-days`.

Criterios de aceptacion

- Listado de citas via `GET /owner-appointments-list` o hub `GET /my-appointments`.
- Detalle via `GET /owner-appointments/:id` con estado, mascota, veterinario y fecha/hora.
- Agendar envia `POST /owner-appointments` tras seleccionar mascota, vet (`GET /owner-available-vets`) y dia/hora (`GET /owner-available-days`).
- Edicion envia `PATCH /owner-appointments/:id` respetando reglas del backend.
- Confirmar asistencia usa `POST /owner-appointments/:id/confirm`.
- Cancelar usa `POST /owner-appointments/:id/cancel` con confirmacion previa.
- Las citas pasadas y proximas son distinguibles en la UI.
- No se permiten slots no disponibles retornados por el API.

RF-OWN-07 — Pagos del dueño

Descripcion

El dueño debe consultar sus pagos pendientes o realizados y registrar comprobante de pago.

Prioridad

Alta

Justificacion

Cierra el ciclo de servicios facturables. Rutas: `GET /my-payments`, `GET /owner-payments-list`, `POST /owner-payments/:id/register`.

Criterios de aceptacion

- Listado via `GET /owner-payments-list` o hub `GET /my-payments`.
- Cada pago muestra monto, concepto, estado y fecha de vencimiento si aplica.
- Registrar pago envia `POST /owner-payments/:id/register` con datos y comprobante requeridos.
- Tras registro exitoso, el estado del pago se actualiza en listado y detalle.

- Pagos ajenos al usuario autenticado no son accesibles.

RF-OWN-08 — Historial medico, consultas y resumen

Descripcion

El dueño debe consultar el historial medico global y por mascota, ver consultas, exportar recetas y marcar tratamientos completados.

Prioridad

Alta

Justificacion

Transparencia clinica y continuidad del cuidado. Rutas: `GET /my-medical-history`, `GET /my-pets/:pet\_id/medical-history`, `GET /owner-medical-summary`, `GET /pets/:pet\_id/consultations`, `GET /owner/consultations/:id/export\_recipe`, `POST /owner/consultations/:id/complete\_treatment`.

Criterios de aceptacion

- Hub de historial via `GET /my-medical-history` y resumen via `GET /owner-medical-summary`.
- Historial por mascota via `GET /my-pets/:pet\_id/medical-history`.
- Listado de consultas via `GET /pets/:pet\_id/consultations`.
- Exportar receta dispara `GET /owner/consultations/:id/export\_recipe` y permite abrir o compartir el archivo resultante.
- Completar tratamiento envia `POST /owner/consultations/:id/complete\_treatment` con confirmacion en UI.
- Solo se muestran registros de mascotas del dueño autenticado.

RF-OWN-09 — Perfil medico, vacunas y desparasitaciones

Descripcion

El dueño debe consultar el perfil medico base de su mascota, el historial de vacunas y desparasitaciones.

Prioridad

Media

Justificacion

Informacion preventiva de salud animal. Rutas: `GET /pets/:pet\_id/medical\_profile`, `GET /pets/:pet\_id/vaccinations`, `GET /pets/:pet\_id/dewormings`.

Criterios de aceptacion

- Perfil medico visible via `GET /pets/:pet\_id/medical\_profile` (alergias, peso, notas segun API).
- Vacunas listadas via `GET /pets/:pet\_id/vaccinations` con fechas y tipo.

- Desparasitaciones listadas via `GET /pets/:pet\_id/dewormings`.
- La UI indica proximas dosis o vencimientos cuando el backend los calcule.

RF-OWN-10 — Reportes medicos y examenes de laboratorio (dueno)

Descripcion

El dueno debe consultar reportes medicos en PDF, subir archivos de laboratorio y consultar resultados cuando el flujo lo permita.

Prioridad

Media

Justificacion

Acceso documental y participacion en estudios clinicos. Rutas: `GET /pets/:pet\_id/medical\_reports`, `GET /pets/:pet\_id/medical\_reports/:id`, `GET /pets/:pet\_id/medical\_reports/:id/download\_pdf`, `POST /owner/lab\_exams/:id/files`, `PATCH /owner/lab\_exams/:id/results`.

Criterios de aceptacion

- Listado de reportes via `GET /pets/:pet\_id/medical\_reports`.
- Detalle via `GET /pets/:pet\_id/medical\_reports/:id`.
- Descarga PDF via `GET /pets/:pet\_id/medical\_reports/:id/download\_pdf` con apertura en visor o app externa.
- Subida de archivos de lab via `POST /owner/lab\_exams/:id/files`.
- Actualizacion de resultados via `PATCH /owner/lab\_exams/:id/results` cuando corresponda al rol dueno.
- Archivos adjuntos muestran progreso de carga y errores de tamano o formato.

RF-OWN-11 — Patrocinios del dueno

Descripcion

El dueno autenticado debe ver el listado de sus patrocinios activos desde su area personal.

Prioridad

Baja

Justificacion

Continuidad del flujo de donaciones en sesion autenticada. Ruta: `GET /my-sponsorships`.

Criterios de aceptacion

- Pantalla accesible desde dashboard o menu del dueno.
- Consume `GET /my-sponsorships` y lista solo patrocinios del usuario.
- Desde cada item se navega al detalle reutilizando RF-PUB-04.

RF-ADM-01 — Gestion de mascotas (administrador)**Descripcion**

El staff autorizado debe listar, crear, editar, eliminar mascotas clinicas y gestionar sus fotos desde el panel admin movil.

Prioridad

Alta

Justificacion

Operacion diaria de la clinica. Rutas: `GET /admin/pets-list`, `GET /admin/pets/:id`, `POST /admin/pets`, `PATCH /admin/pets/:id`, `PUT /admin/pets/:id`, `DELETE /admin/pets/:id`, `POST /admin/pets/:id/photo`, `DELETE /admin/pets/:id/photo`.

Criterios de aceptacion

- Listado con busqueda via `GET /admin/pets-list`.
- CRUD completo sobre `/admin/pets` segun permisos del rol.
- Gestion de foto via POST/DELETE en `/admin/pets/:id/photo`.
- Solo usuarios admin autenticados acceden a estas pantallas.
- Acciones destructivas requieren confirmacion.

RF-ADM-02 — Gestion de adopciones (administrador)**Descripcion**

El staff debe listar solicitudes o registros de adopcion, ver detalle y crear nuevos registros de adopcion.

Prioridad

Media

Justificacion

Gestion interna del programa de adopcion. Rutas: `GET /admin/adoptions-list`, `GET /admin/adoptions/:id`, `POST /admin/adoptions`.

Criterios de aceptacion

- Listado via `GET /admin/adoptions-list`.
- Detalle via `GET /admin/adoptions/:id`.
- Alta via `POST /admin/adoptions` con validacion de campos.
- Estados de adopcion se muestran segun respuesta del API.

RF-ADM-03 — Consultas clinicas (administrador)**Descripcion**

El staff debe gestionar consultas: listar, crear, editar, eliminar, exportar receta y completar tratamientos.

Prioridad

Alta

Justificacion

Nucleo del trabajo veterinario en la app. Rutas: `GET /admin/consultations-list`, `GET /admin/consultations/:id`, `POST /admin/consultations`, `PATCH /admin/consultations/:id`, `PUT /admin/consultations/:id`, `DELETE /admin/consultations/:id`, `GET /admin/consultations/:id/export\_recipe`, `POST /admin/consultations/:id/complete\_treatment`.

Criterios de aceptacion

- Listado filtrable via `GET /admin/consultations-list`.
- Formulario de consulta envia POST o PATCH segun creacion o edicion.
- Detalle muestra notas clinicas, mascota, veterinario y estado.
- Exportar receta via `GET /admin/consultations/:id/export\_recipe`.
- Completar tratamiento via `POST /admin/consultations/:id/complete\_treatment`.
- Eliminar consulta via DELETE con confirmacion.

RF-ADM-04 — Recetas y exámenes de laboratorio (administrador)**Descripcion**

Dentro de una consulta, el staff debe gestionar prescripciones y exámenes de laboratorio, incluyendo archivos adjuntos.

Prioridad

Alta

Justificacion

Documentacion clinica completa en consulta. Rutas prescriptions y lab_exams bajo `/admin/consultations/:consultation\_id/`.

Criterios de aceptacion

- Listar, crear, editar y eliminar prescripciones via endpoints `/admin/consultations/:consultation\_id/prescriptions`.
- Listar, crear, editar y eliminar exámenes via `/admin/consultations/:consultation\_id/lab\_exams`.
- Subir archivos via `POST .../lab\_exams/:id/files`.
- Eliminar archivo via `DELETE .../lab\_exams/:id/files/:file\_id`.
- Cambios en receta/lab se reflejan al volver al detalle de consulta.

RF-ADM-05 — Vacunas, desparasitaciones y esquemas**Descripcion**

El staff debe administrar registros de vacunacion, desparasitacion y esquemas programados, incluyendo marcar items completados.

Prioridad

Alta

Justificacion

Control preventivo de salud animal. Rutas: `/admin/vaccinations-\*`, `/admin/dewormings-\*`, `/admin/vaccination-schedules-\*`, `POST /admin/vaccination\_schedules/:id/complete\_item`.

Criterios de aceptacion

- CRUD de vacunas via endpoints `/admin/vaccinations` y listado `-list`.
- CRUD de desparasitaciones via endpoints `/admin/dewormings`.
- CRUD de esquemas via `/admin/vaccination\_schedules`.
- Completar item de esquema via `POST /admin/vaccination\_schedules/:id/complete\_item`.
- Listados muestran mascota asociada y fechas relevantes.

RF-ADM-06 — Agenda y citas (administrador)

Descripcion

El staff debe gestionar la agenda: listar citas, crear y editar citas usando disponibilidad de veterinarios, dias y franjas horarias.

Prioridad

Alta

Justificacion

Coordinacion operativa de la clinica. Rutas: `GET /admin/appointments-list`, `GET /admin/appointments/:id`, `GET /admin/appointments/available-days`, `GET /admin/appointments/available-vets`, `GET /admin/appointments/time-slots`, `POST /admin/appointments`, `PATCH /admin/appointments/:id`, `PUT /admin/appointments/:id`, `DELETE /admin/appointments/:id`.

Criterios de aceptacion

- Listado/calendario via `GET /admin/appointments-list`.
- Creacion usa vets, dias y slots de los endpoints de disponibilidad antes de `POST /admin/appointments`.
- Edicion y cancelacion via PATCH/PUT/DELETE en `/admin/appointments/:id`.
- Detalle muestra dueño, mascota, servicio y sala si aplica.
- Conflictos de horario retornados por el API se muestran al usuario.

RF-ADM-07 — Pagos (administrador)

Descripcion

El staff debe listar pagos, ver detalle, registrar pagos recibidos y actualizar su estado.

Prioridad

Alta

Justificacion

Control financiero de servicios. Rutas: `GET /admin/payments-list`, `GET /admin/payments/:id`, `POST /admin/payments/:id/register`, `PATCH /admin/payments/:id`, `PUT /admin/payments/:id`.

Criterios de aceptacion

- Listado via `GET /admin/payments-list` con filtros por estado si el API los soporta.
- Detalle via `GET /admin/payments/:id`.
- Registro de pago via `POST /admin/payments/:id/register`.
- Actualizacion de estado o datos via PATCH/PUT.
- Comprobantes adjuntos son visibles cuando el backend los incluya.

RF-ADM-08 — Perfil medico de mascotas (administrador)**Descripcion**

El staff debe consultar y editar el perfil medico base de una mascota bajo administracion.

Prioridad

Media

Justificacion

Datos clinicos de referencia para consultas. Rutas: `GET /admin/pets/:pet\_id/medical\_profile`, `PATCH /admin/pets/:pet\_id/medical\_profile`, `PUT /admin/pets/:pet\_id/medical\_profile`.

Criterios de aceptacion

- Visualizacion via GET del perfil medico.
- Edicion via PATCH o PUT con validacion de campos clinicos.
- Cambios persisten y se reflejan en consultas posteriores.

RF-ADM-09 — Mascotas de duenos (administrador)**Descripcion**

El staff debe poder crear y editar mascotas asociadas a un dueno especifico, incluyendo foto.

Prioridad

Media

Justificacion

Alta asistida en recepcion o consulta. Rutas: `GET /admin/owners/:owner\_id/pets/:id`, `POST /admin/owners/:owner\_id/pets`, `PATCH /admin/owners/:owner\_id/pets/:id`, `PUT /admin/owners/:owner\_id/pets/:id`, `POST/DELETE ../photo`.

Criterios de aceptacion

- Creacion de mascota para dueño via `POST /admin/owners/:owner\_id/pets`.
- Edicion via PATCH/PUT en ruta anidada con owner_id.
- Gestion de foto via endpoints de photo del recurso.
- Detalle accesible via GET con datos del dueño y la mascota.

RF-ADM-10 — Reportes medicos (administrador)

Descripcion

El staff debe listar reportes medicos, generarlos, importarlos, exportarlos en multiples formatos y enviarlos por email.

Prioridad

Media

Justificacion

Documentacion formal y comunicacion con dueños. Rutas: `/admin/medical\_reports` (index, show, create, generate_pdf, import, export_csv/json/pdf, email).

Criterios de aceptacion

- Listado via `GET /admin/medical\_reports`.
- Detalle via `GET /admin/medical\_reports/:id`.
- Generar PDF via `GET /admin/medical\_reports/generate\_pdf` con parametros requeridos.
- Importar via `POST /admin/medical\_reports/import`.
- Exportar CSV, JSON y PDF via endpoints `export\_\*`.
- Enviar por email via `POST /admin/medical\_reports/:id/email`.
- Archivos generados se pueden abrir o compartir desde el dispositivo.

Parte 2 — Requisitos no funcionales

Los RNF siguen el formato `RNF-{Categoria}-{NNN}: {Titulo}` con enunciado directo. Categorías: **ARQ** (arquitectura), **MOB** (movil), **SEC** (seguridad), **UX** (experiencia), **REL** (confiabilidad), **PERF** (rendimiento), **ARC** (archivos), **PRI** (privacidad), **OBS** (observabilidad), **CMP** (compatibilidad), **MAINT** (mantenibilidad).

RNF-ARQ-001: Plataforma movil Android

La aplicacion debe desarrollarse con **Expo SDK 56**, **React Native** y **TypeScript**, como cliente nativo **solo para Android**. El codigo de pantallas y servicios debe estar tipado

de forma estricta. Los flujos principales no deben depender de contenedores web embebidos. La version de Expo debe mantenerse alineada con la documentacion oficial v56 (`AGENTS.md`, `docs/arq.md`). No se incluyen targets, builds ni dependencias especificas de iOS.

RNF-ARQ-002: Integracion con API Rails

La app debe consumir exclusivamente las **157 rutas** documentadas en `mobile-routes.md`, sin reimplementar logica de negocio en el cliente. Debe existir un cliente HTTP centralizado con base URL configurable por entorno (desarrollo, staging, produccion). Los payloads y respuestas deben estar tipados segun los contratos del API Rails existente.

RNF-MOB-001: Mobile First

La aplicacion debe disenarse y construirse con enfoque **mobile first**: layouts, navegacion, tipografia y patrones de interaccion pensados primero para pantallas de telefono (una mano, orientacion vertical, safe areas). Las adaptaciones a tablet o web, si existen en el futuro, deben ser extensiones del diseno movil base y no condicionar la arquitectura de la app. Toda pantalla documentada en `mobile-screens.md` debe priorizar legibilidad y acciones principales en viewports desde **320 px** de ancho logico.

RNF-MOB-002: Auto-sincronizacion bajo consentimiento del usuario

La aplicacion debe ser capaz de **auto-sincronizarse** con el backend cuando detecte conexion activa por **WiFi o datos moviles, si y solo si el usuario lo ha autorizado** previamente en ajustes (opt-in explicito, revocable en cualquier momento).

Comportamiento esperado:

- Sin autorizacion del usuario, **ninguna** sincronizacion automatica en segundo plano debe ejecutarse; solo sincronizacion manual iniciada por el usuario.
- Con autorizacion activa, al detectar conectividad WiFi o datos, la app debe procesar la cola local de operaciones pendientes (formularios guardados, uploads, acciones diferidas) y refrescar datos stale segun prioridad definida por negocio.
- El usuario debe poder elegir, como minimo, si permite sincronizar solo con **WiFi** o tambien con **datos moviles**.
- Toda sincronizacion automatica debe ser visible mediante indicador discreto de estado (sincronizando / al dia / pendiente / error) sin interrumpir el flujo actual salvo conflicto que requiera decision.

RNF-SEC-001: Seguridad de sesion y almacenamiento de tokens

Los tokens de autenticacion y demas secretos deben almacenarse en el almacenamiento seguro del sistema operativo cuando este disponible, **nunca** en

almacenamiento plano sin cifrar. Toda comunicacion con el backend debe realizarse por **HTTPS**. El refresh de sesion debe gestionarse via ``POST /auth/refresh`` sin exponer tokens en logs de produccion. Al cerrar sesion (``DELETE /auth/logout``) deben eliminarse todos los secretos y datos de sesion locales.

RNF-SEC-002: Control de acceso por rol y dominio

La navegacion y las llamadas API deben restringirse segun rol: **publico**, **dueno (owner)** o **administrador (admin)**. Un dueno autenticado no debe acceder a pantallas ni endpoints ``/admin/*``. El staff admin solo accede al dominio Admin segun permisos retornados por ``GET /auth/current_user``. Los intentos no autorizados deben redirigir a login o informar error **403** de forma clara.

RNF-UX-001: Experiencia de usuario movil

La interfaz debe cumplir patrones moviles de produccion en Android: areas tactiles minimas de **48x48 dp** (Material), inputs con **font-size ≥ 16 px**, estados de carga / vacio / error en todo listado o formulario, navegacion por tabs y stacks segun ``mobile-screens.md``, y respeto de **safe areas** y barras del sistema (status bar, navigation bar, recortes de pantalla). Los mensajes al usuario deben estar en espanol claro, sin stack traces ni jerga tecnica.

RNF-UX-002: Design system y tema claro/oscuro

La UI debe implementar los tokens centralizados de ``mobile-design-tokens.css`` y ``src/theme/tokens.ts``, incluyendo **modo claro y modo oscuro** coherentes con ``mobile-ui-components.md``. El tema puede seguir la preferencia del sistema o una seleccion manual del usuario.

RNF-REL-001: Tolerancia a fallos

El sistema debe manejar los errores comunes con elegancia — interrupciones de red durante el envio de datos, timeouts, respuestas **5xx** o token expirado — e informar al usuario adecuadamente, intentando **preservar los datos ingresados** siempre que sea posible (borrador local, cola de reintentos, no descartar formularios completados). Los errores **422** de validacion deben senalar el campo afectado. Los **401** deben disparar refresh de sesion o re-autenticacion segun corresponda.

RNF-REL-002: Copia de seguridad local y recuperacion

Objetivo de Punto de Recuperacion (RPO): En caso de cierre inesperado de la app o fallo del dispositivo, la perdida de datos no sincronizados encolados localmente no debe exceder el **ultimo guardado automatico de borrador** (intervalo maximo recomendado: **5 minutos** mientras el usuario edita, o al perder foco del formulario).

Objetivo de Tiempo de Recuperacion (RTO): Tras reiniciar la app, el usuario debe poder retomar operaciones pendientes o borradores recuperables en un plazo maximo de **30 segundos** desde el arranque, sin reingresar datos ya capturados localmente.

Los datos clinicos descargados no deben depender de cache permanente sin cifrar; la recuperacion aplica a **cola de sincronizacion, borradores y estado de sesion** autorizado.

RNF-REL-003: Operacion offline controlada

Sin conexion, la app debe permitir consultar la **ultima copia local** de datos ya sincronizados (cuando exista) y encolar acciones de escritura para envio posterior. Las acciones encoladas deben mostrarse al usuario como **pendientes de sincronizacion**. No debe simular exito de operaciones que no hayan sido confirmadas por el servidor.

RNF-PERF-001: Rendimiento en red movil

Las transiciones entre pantallas principales deben percibirse en **menos de 1 segundo** con conexion normal. Las operaciones largas (upload/download) no deben bloquear la UI; deben mostrar progreso y permitir cancelacion cuando el SO lo permita. Los listados deben soportar paginacion si el API la ofrece.

RNF-ARC-001: Manejo de archivos y multimedia

La app debe soportar captura y subida de imagenes (fotos de mascota, comprobantes de pago) y descarga o visualizacion de **PDFs medicos** y archivos de laboratorio, conforme a las rutas de upload/download del API. Debe respetar limites de tamano y tipo MIME del backend. Los permisos de camara, galeria y almacenamiento deben solicitarse en runtime con mensaje contextual.

RNF-PRI-001: Privacidad y datos de salud animal

Los datos medicos y personales deben tratarse con minima exposicion: sin cache innecesaria en disco, sin logs con informacion identificable en builds de produccion, y con acceso a la politica de privacidad (`GET /pages/privacy`) desde registro y perfil. La eliminacion de cuenta debe purgar datos locales del usuario. La auto-sincronizacion (RNF-MOB-002) nunca debe activarse sin consentimiento explicito del usuario.

RNF-OBS-001: Observabilidad y registro de errores

Los errores no controlados deben registrarse con contexto tecnico en entorno de **desarrollo** unicamente. En produccion, el usuario recibe mensajes accionables; el equipo tecnico puede correlacionar fallos mediante identificadores de error anonimos, sin incluir tokens, contraseñas ni diagnosticos veterinarios completos.

RNF-CMP-001: Compatibilidad de dispositivos Android

La app debe ser compatible con las versiones de **Android** soportadas por Expo SDK 56 segun `app.json` (API level minimo definido en el proyecto). Debe funcionar en telefonos pequenos (320 px de ancho logico) y grandes, con interaccion tactil completa (sin depender de teclado fisico). Debe validarse manualmente en al menos un dispositivo Android fisico y un emulador durante QA.

RNF-MAINT-001: Mantenibilidad y trazabilidad documental

El codigo debe mantener trazabilidad entre requisitos (**RF-***, **RNF-***), rutas API (`mobile-routes.md`), pantallas (`mobile-screens.md`) y componentes (`mobile-ui-components.md`). Los servicios API deben agruparse por dominio en `src/services/`. Cambios en rutas incluidas deben actualizar la documentacion en `docs/`. El demo HTML es referencia visual, no codigo de produccion.

Matriz de trazabilidad resumida

Dominio	Requisitos funcionales	Rutas API (aprox.)
Public	RF-PUB-01 a RF-PUB-05	19
Owner	RF-OWN-01 a RF-OWN-11	54
Admin	RF-ADM-01 a RF-ADM-10	84
No funcionales	RNF-ARQ, MOB, SEC, UX, REL, PERF, ARC, PRI, OBS, CMP, MAINT	—

Documentos relacionados

Documento	Contenido
[mobile-routes.md](./mobile-routes.md)	Catalogo de endpoints
[mobile-screens.md](./mobile-screens.md)	Pantallas por proceso
[mobile-ui-components.md](./mobile-ui-components.md)	Componentes UI
[mobile-demo.html](./mobile-demo.html)	Prototipo visual
[arq.md](./arq.md)	Arquitectura tecnica

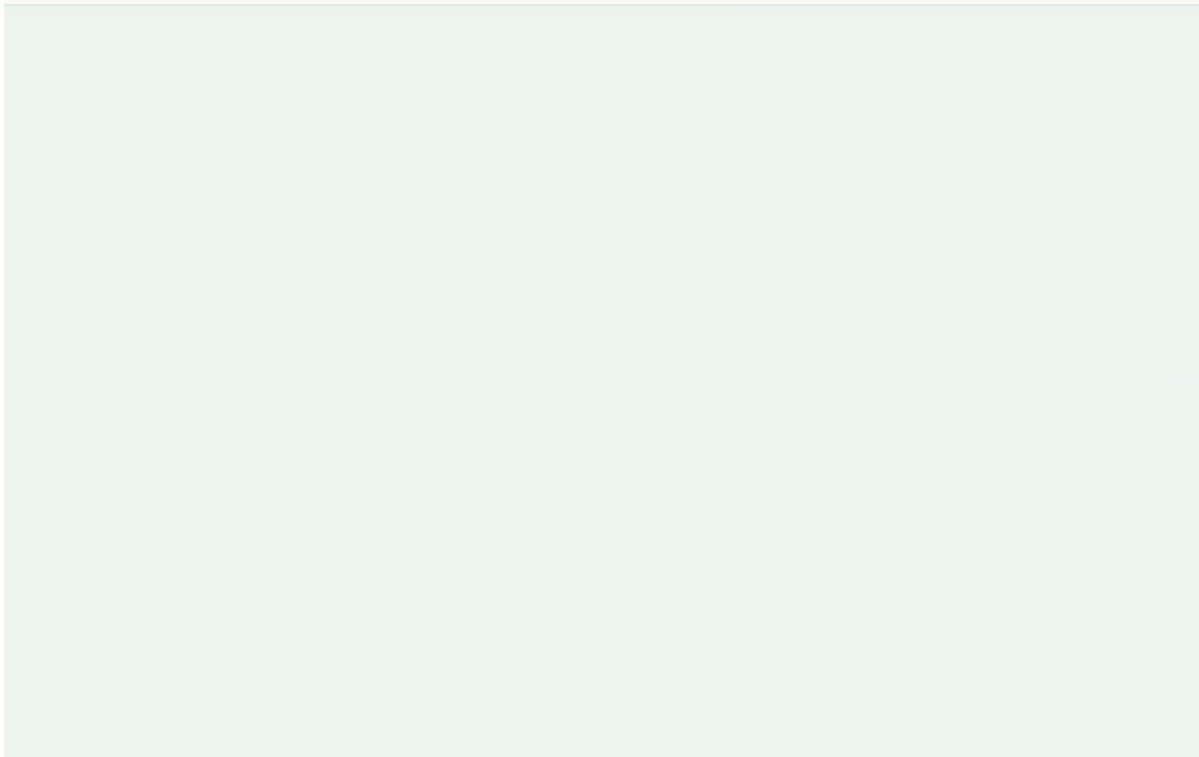
ANEXO B

Demo de interfaz de usuario (Demo UI)

Fuente: docs/PawCare Mobile — Demo UI.pdf — sistema visual con temas claro y oscuro

PawCare Mobile — Demo UI

Solo diseño · Sin funcionalidad · Basado en `mobile-routes.md` · Tokens en `mobile-design-tokens.css`



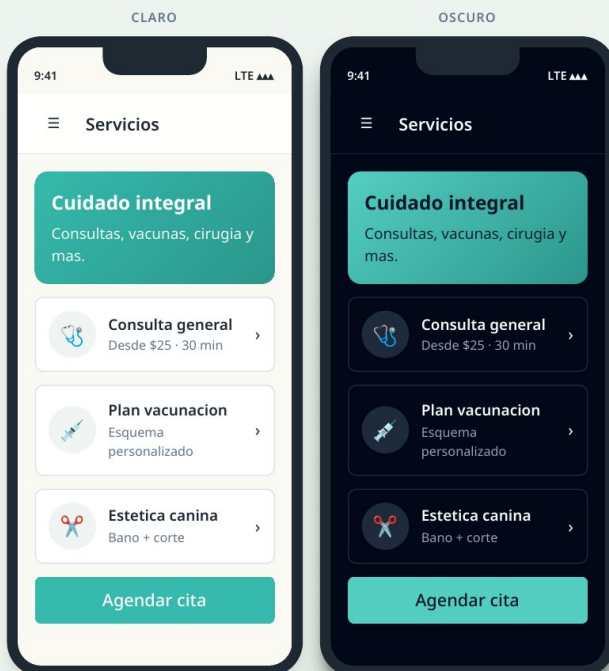
Public — Descubrimiento y compra

Servicios, tienda, adopcion e informacion legal. Sin autenticacion.

Servicios

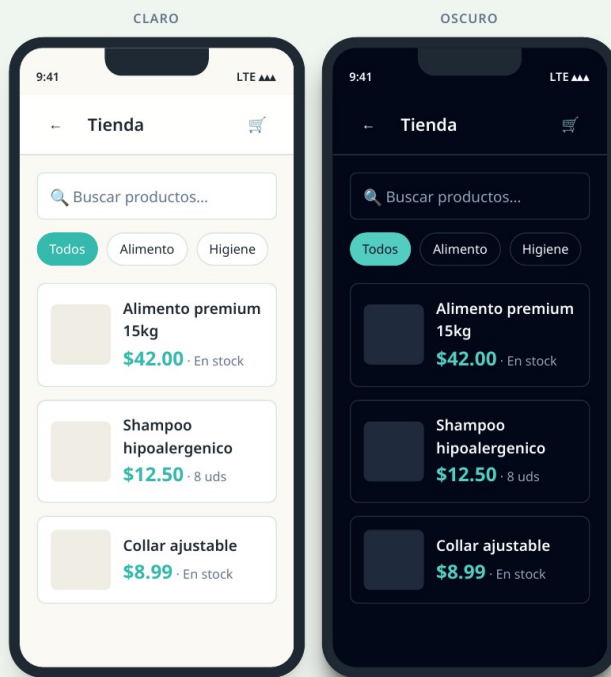
GET /services

Public



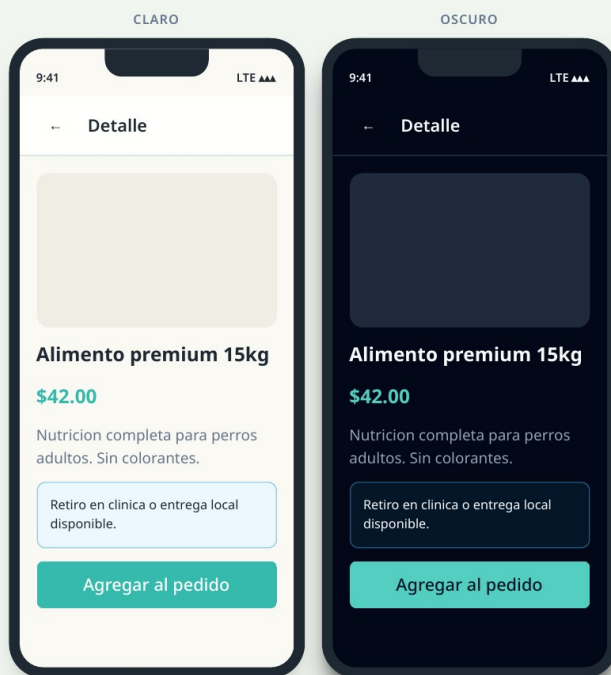
Productos

GET /public/products



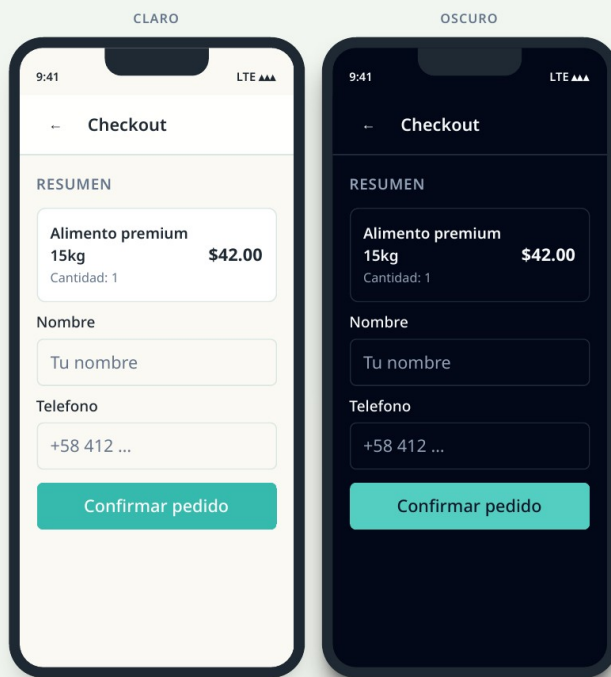
Detalle producto

GET /public/products/:id



Checkout

POST /public/product_orders

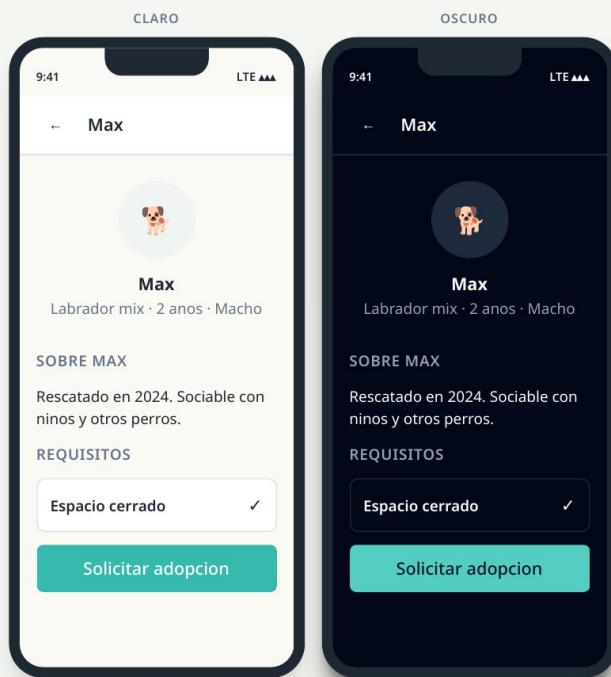
**Adopcion**

GET /adoption · /adoption/pets



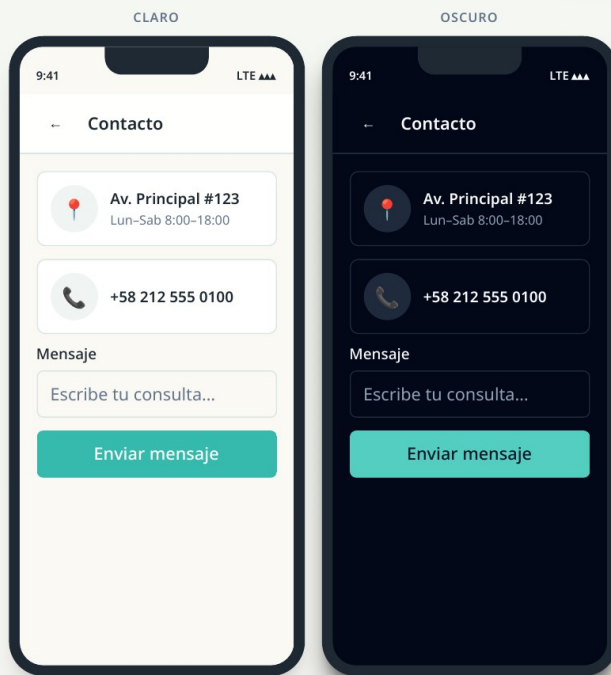
Detalle adopcion

GET /adoption/pets/:id



Contacto

GET /pages/contact

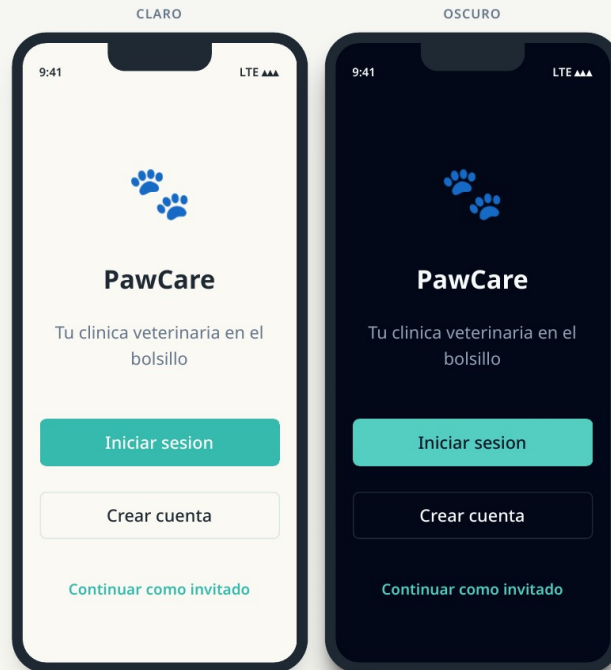


Owner — Autenticacion

Flujo de acceso para dueños de mascota.

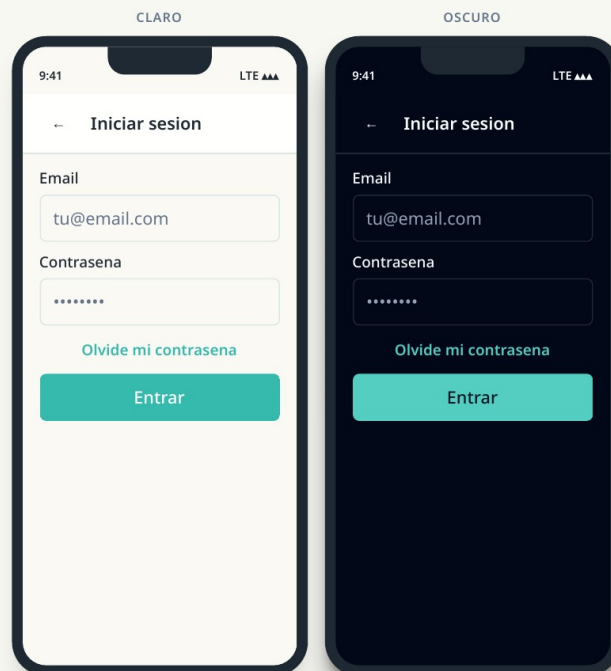
Bienvenida

GET /auth



Login

POST /auth/login



Registro

POST /auth/register_owner

CLARO

9:41 LTE

Crear cuenta

Nombre completo

Maria Gonzalez

Email

maria@email.com

Contraseña

Min. 8 caracteres

Registrarme

Al registrarte aceptas los Terminos y la Política de privacidad.

OSCURO

9:41 LTE

Crear cuenta

Nombre completo

Maria Gonzalez

Email

maria@email.com

Contraseña

Min. 8 caracteres

Registrarme

Al registrarte aceptas los Terminos y la Política de privacidad.

Recuperar contraseña

POST /auth/forgot_password

CLARO

9:41 LTE

Recuperar acceso

Te enviaremos un enlace para restablecer tu contraseña.

Email

tu@email.com

Enviar enlace

OSCURO

9:41 LTE

Recuperar acceso

Te enviaremos un enlace para restablecer tu contraseña.

Email

tu@email.com

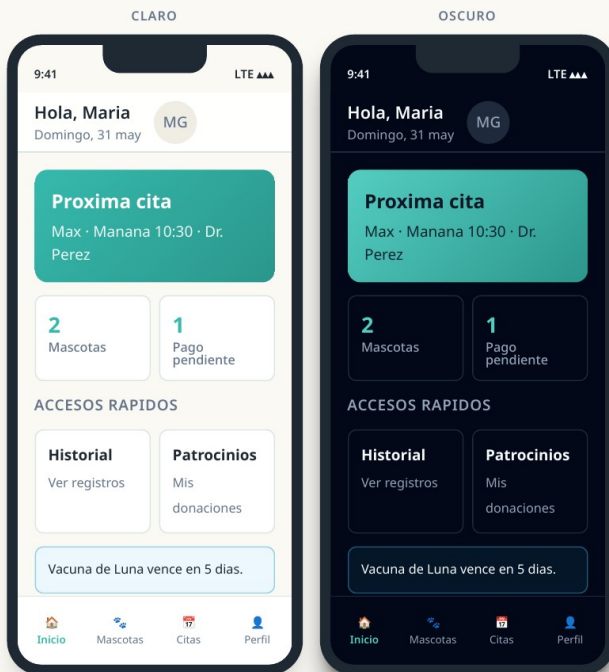
Enviar enlace

Owner — App principal

Dashboard, mascotas, citas, pagos y perfil.

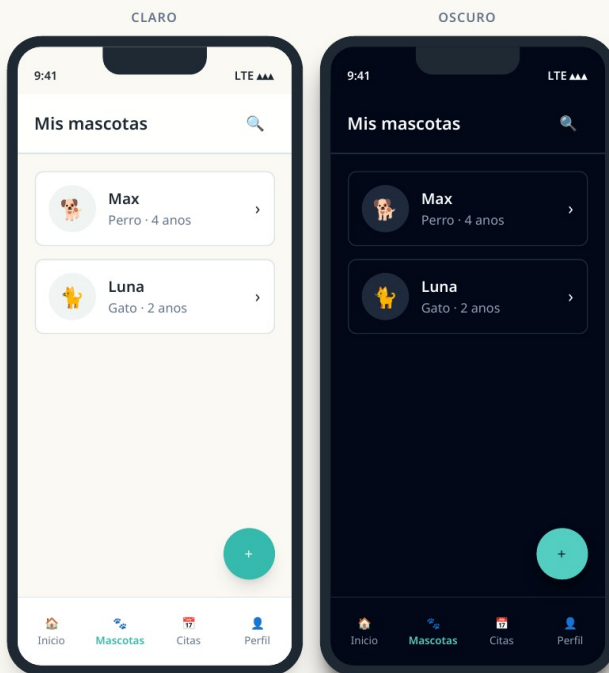
Dashboard

GET /dashboard · /dashboard-summary



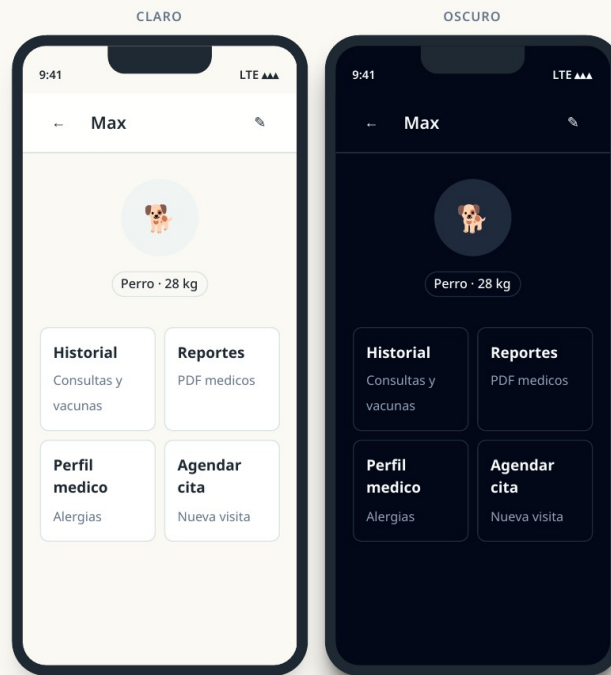
Mis mascotas

GET /my-pets · /pets



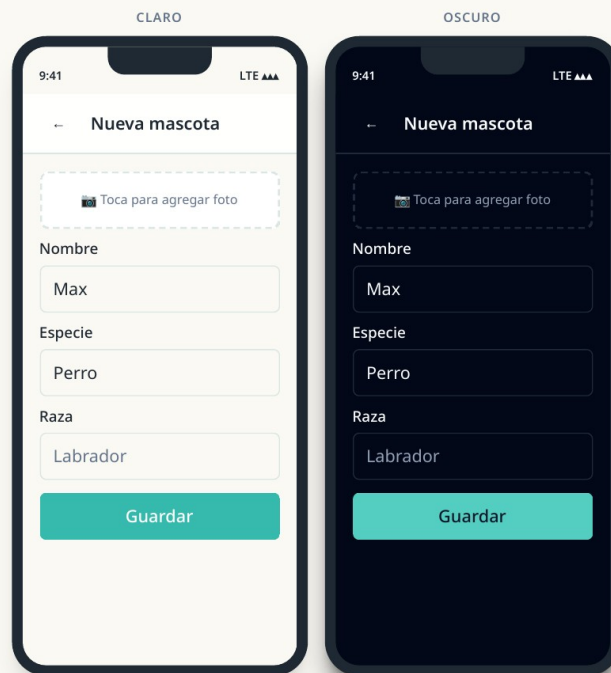
Detalle mascota

GET /pets/:id



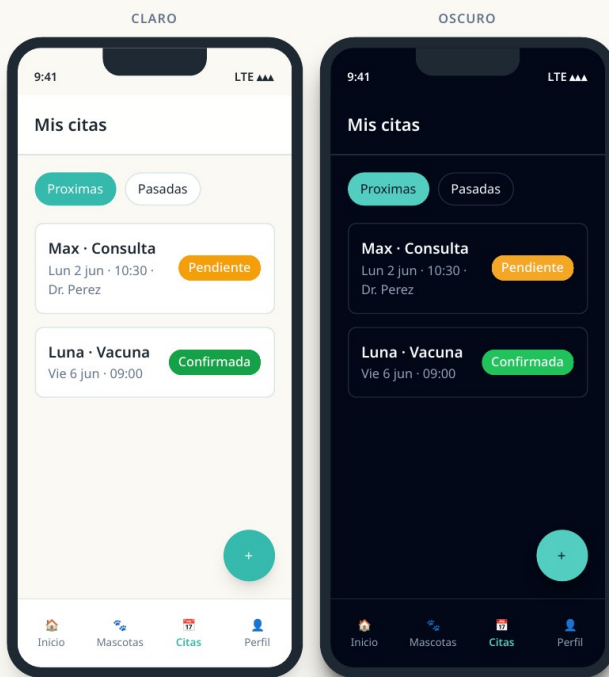
Nueva mascota

POST /pets · POST /pets/:id/photo



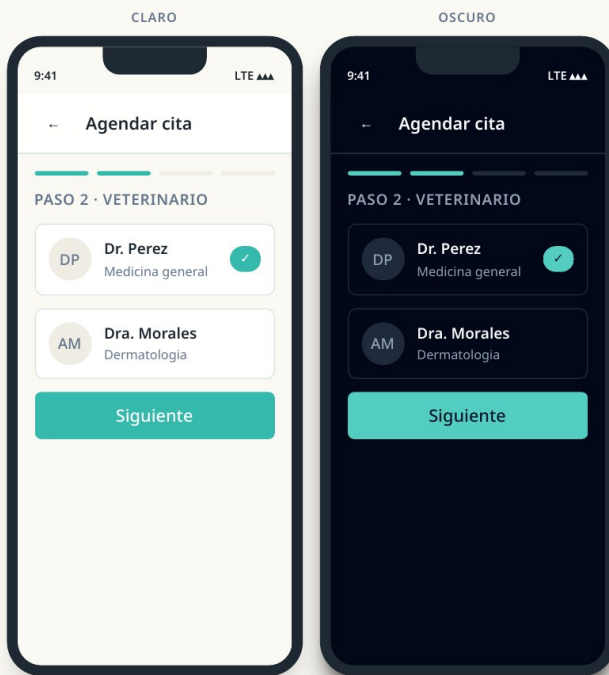
Mis citas

GET /owner-appointments-list



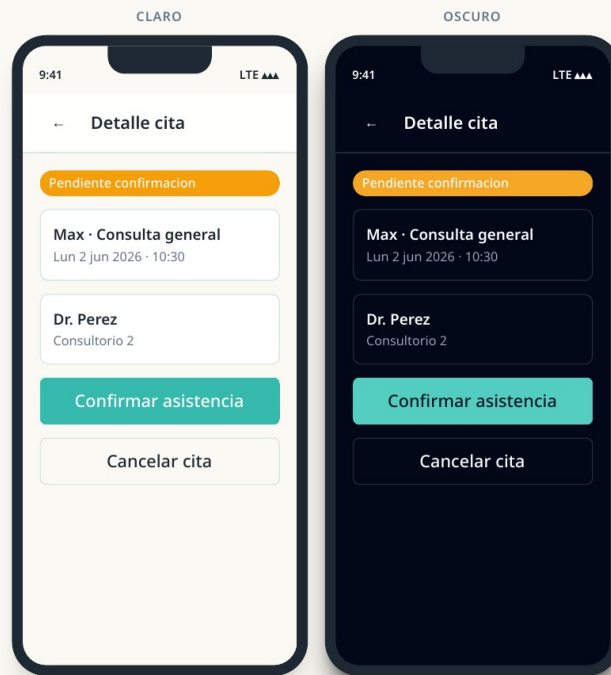
Agendar cita

POST /owner-appointments · availability



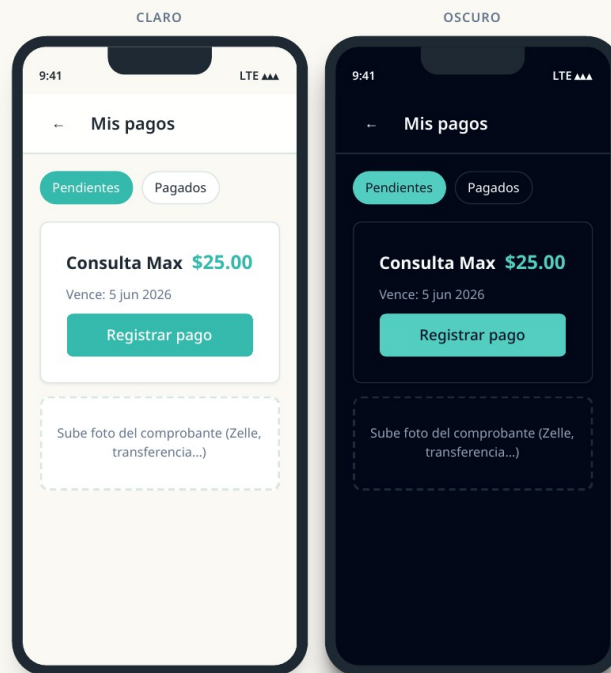
Detalle cita

GET /owner-appointments/:id · confirm/cancel



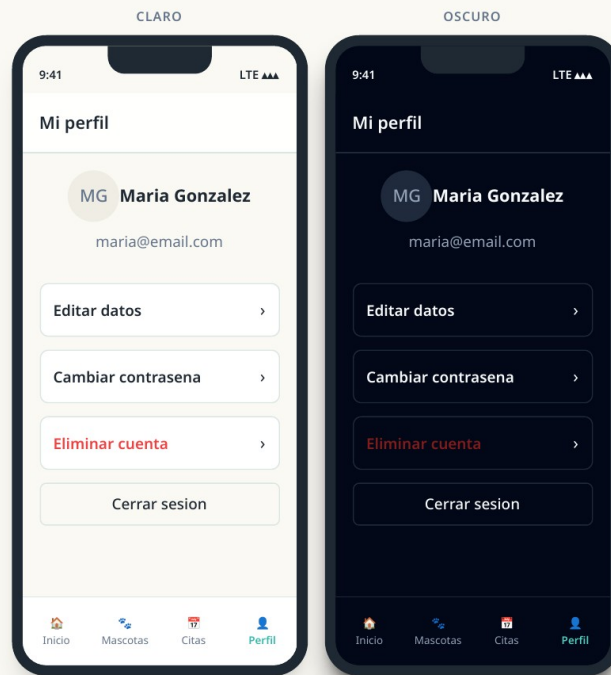
Pagos

GET /owner-payments-list · register



Mi perfil

GET /profile · PATCH /profile

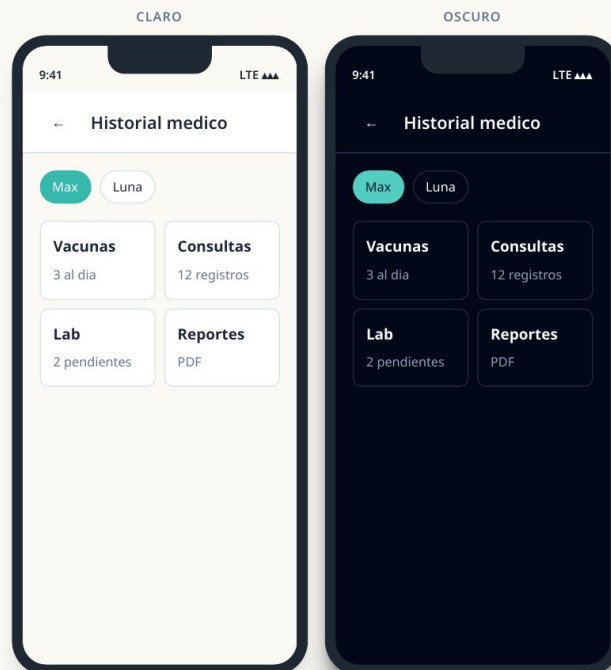


Owner — Historial medico

Consultas, vacunas, laboratorio y reportes por mascota.

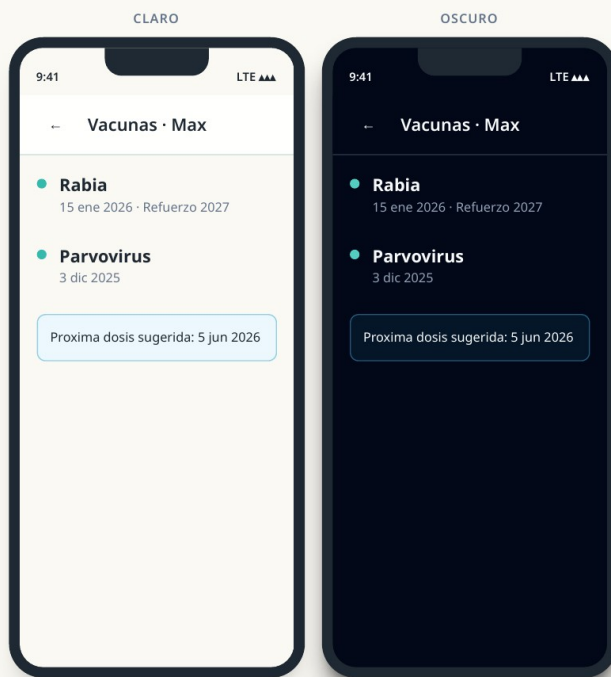
Historial medico

GET /my-medical-history · /owner-medical-summary



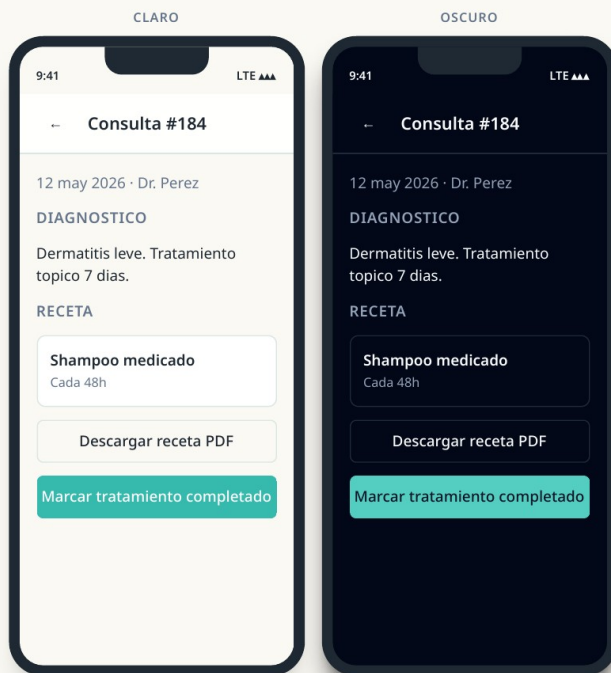
Vacunas

GET /pets/:pet_id/vaccinations



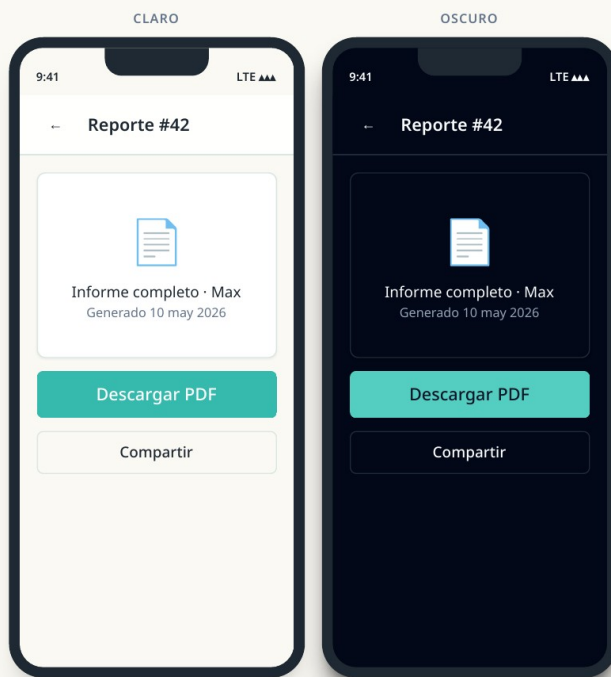
Detalle consulta

GET /pets/:pet_id/consultations · export_recipe



Reporte medico

GET /pets/:pet_id/medical_reports/:id



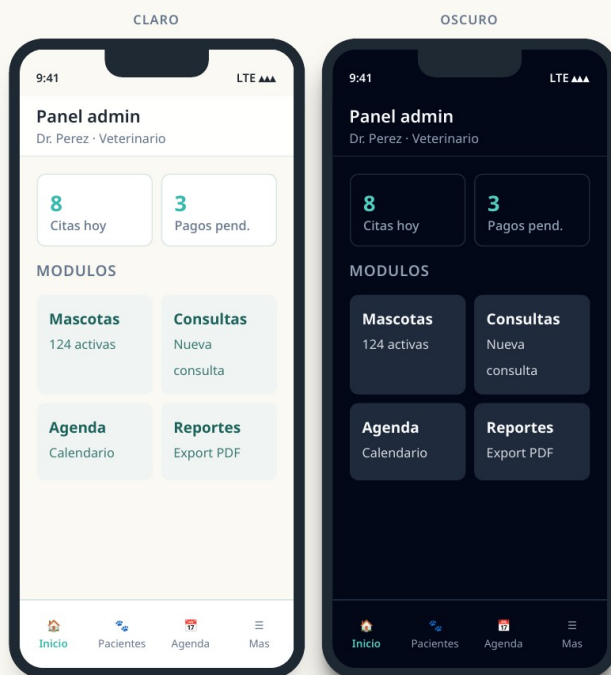
Admin — Panel administrativo

Flujos de staff: pacientes, consultas, agenda, pagos y reportes.

Admin inicio

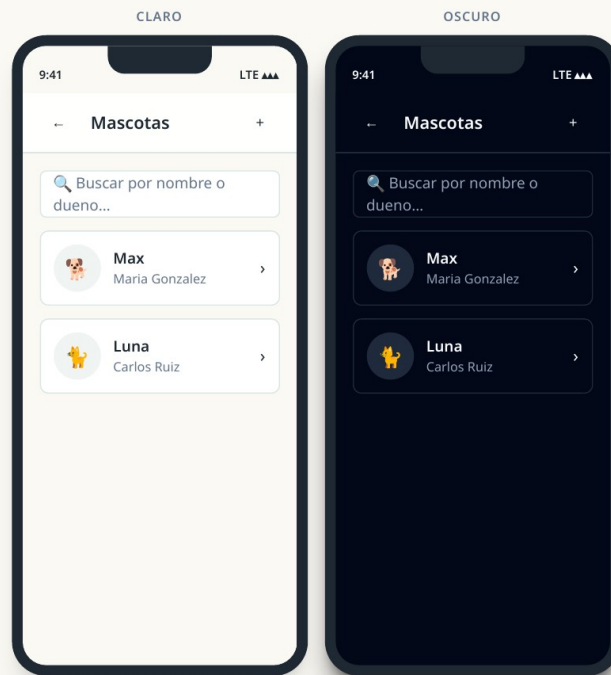
—

Admin



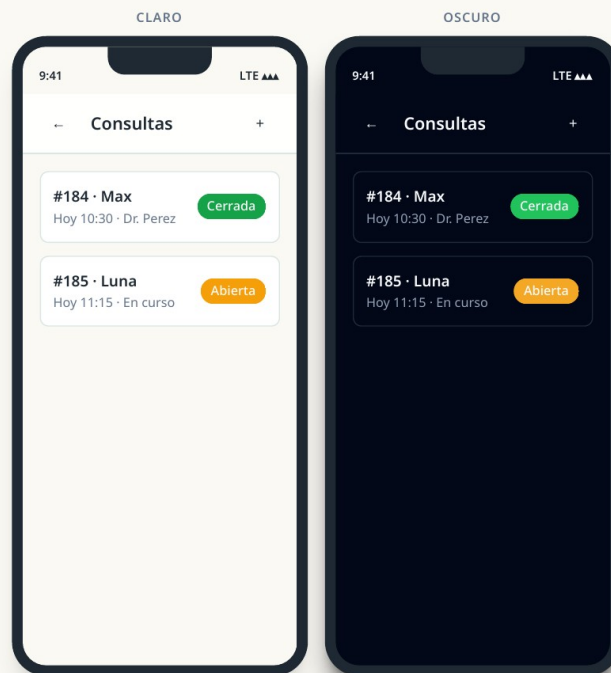
Mascotas admin

GET /admin/pets-list



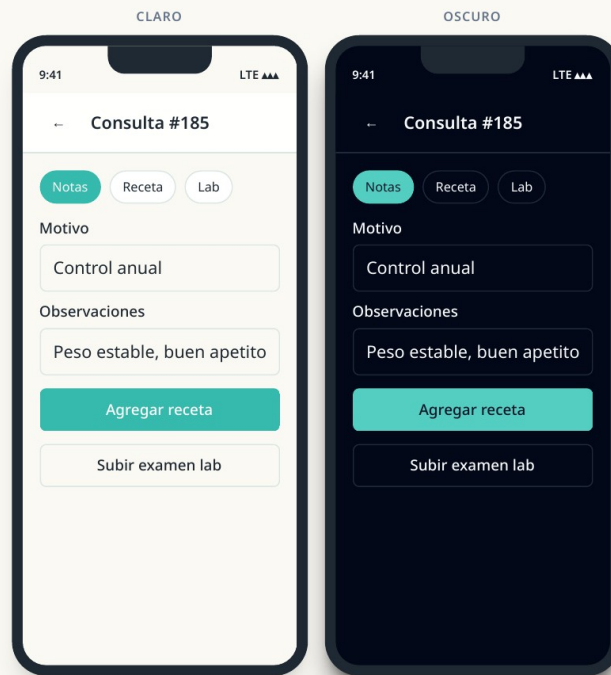
Consultas admin

GET /admin/consultations-list



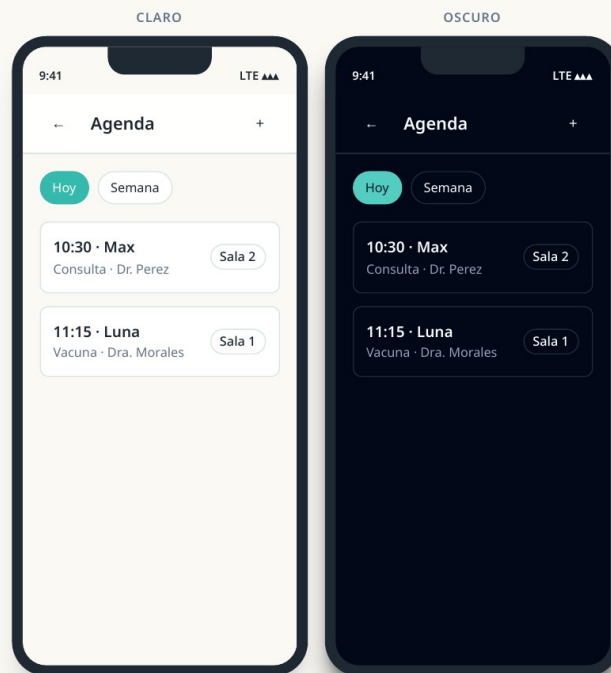
Detalle consulta admin

prescriptions · lab_exams



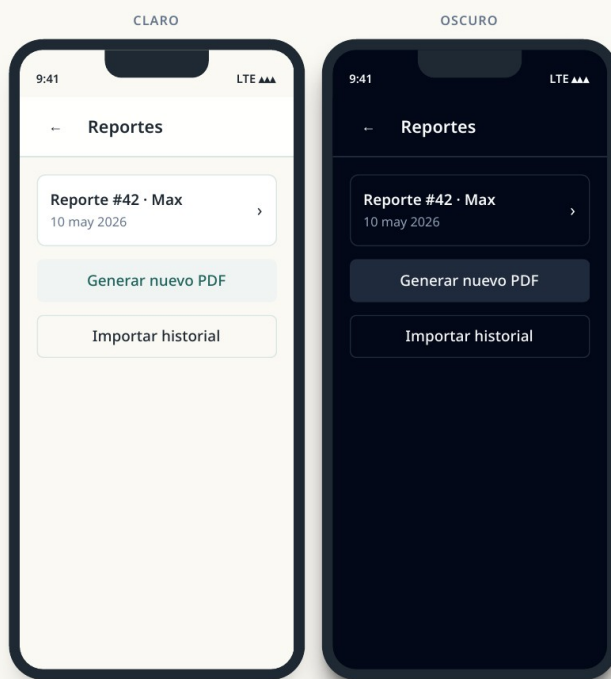
Agenda admin

GET /admin/appointments-list



Reportes medicos admin

GET /admin/medical_reports



ANEXO C

Diagramas de arquitectura

Fuente: docs/PawCare Mobile — Diagramas de Arquitectura.pdf (v4.0, julio 2026)

PawCare Mobile

Diagramas de arquitectura — aplicacion movil (Expo SDK 56 · React Native · TypeScript)

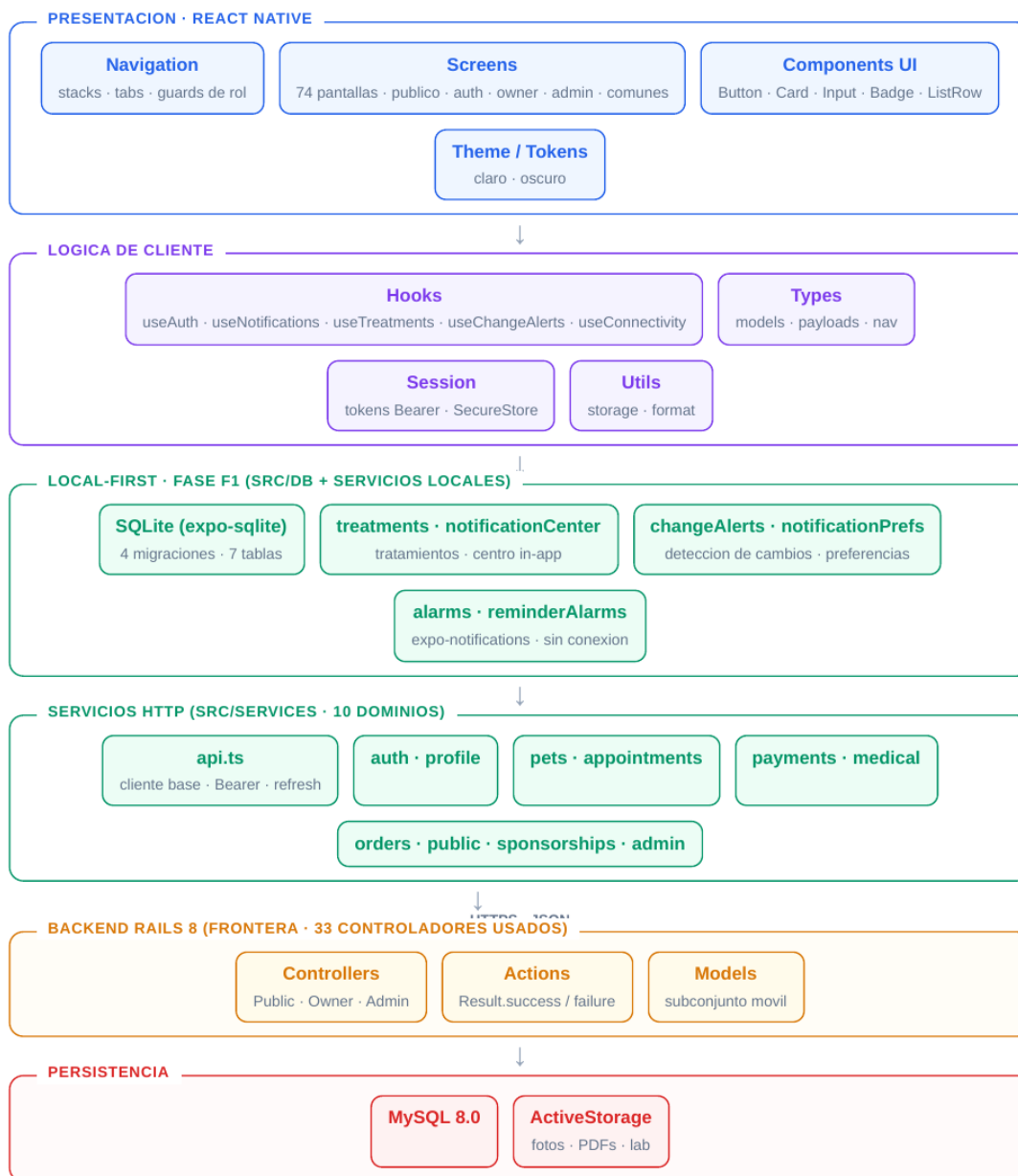
Alcance: este documento cubre **unicamente lo que consume la app movil** — las **157 rutas** de `mobile-routes.md` (Public 19 · Owner 54 · Admin 84), repartidas en **33 controladores** del backend Rails. La autenticacion de la app usa las rutas dedicadas `/auth/mobile/*` (sesion Bearer, fase F4), equivalentes moviles del grupo auth.

Excluido (solo web): metricas, turnos/shifts, modulo financiero, gestion de contenido y campanas de email, administracion de staff, movimientos de stock, categorias de servicio y configuracion. Esos endpoints existen en el backend pero la app movil no los usa.

Estado implementado: 74 pantallas con datos reales · 10 servicios de dominio HTTP · base local SQLite (fase F1 del ADR 0001: 4 migraciones, 7 tablas) · sistema de notificaciones y alarmas de medicacion local-first (sprint 4).

1. Diagrama de capas

Flujo vertical cliente → backend. La app movil es el cliente; el backend Rails se muestra solo como frontera de integracion.



2. Diagrama de flujo de una petición

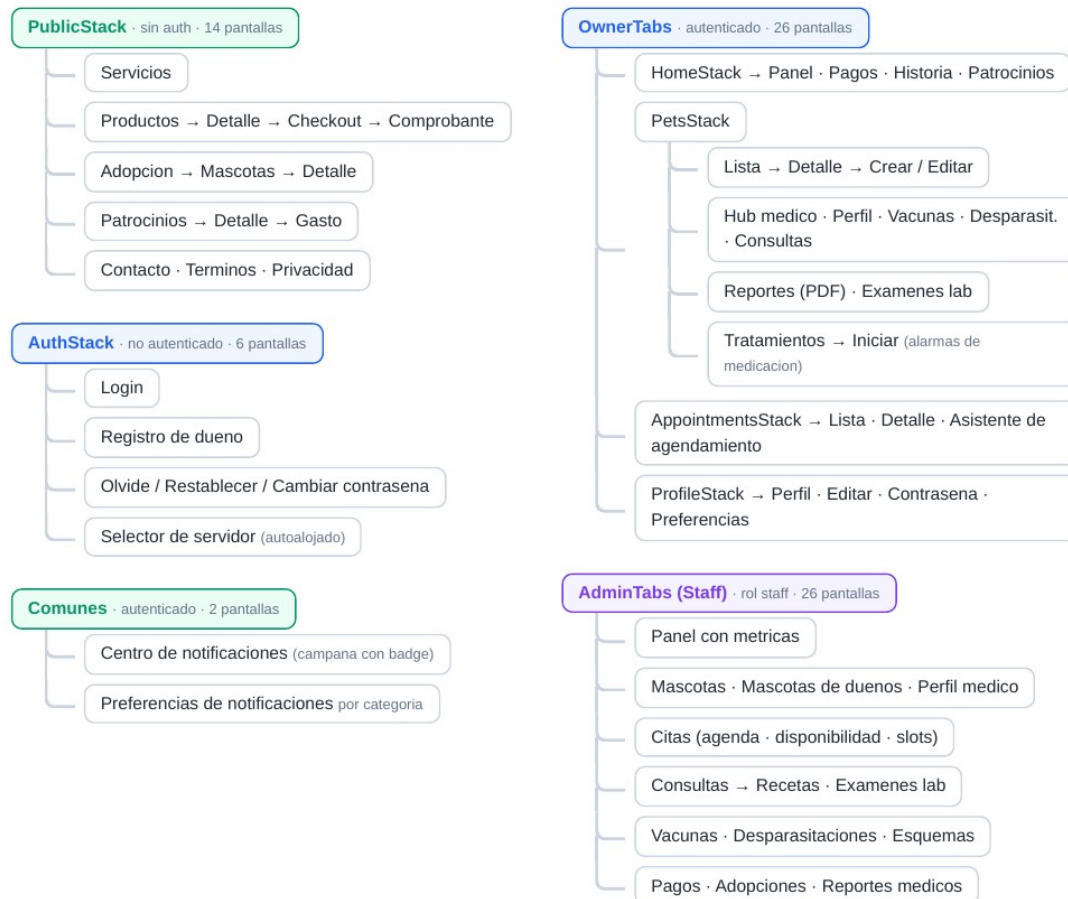
Ciclo de vida de una acción del usuario (ej. agendar cita, crear mascota).



Manejo de errores y operacion offline (RNF-REL / RNF-MOB-002). Estado actual: F1 entregada; F2-F4 como continuidad.

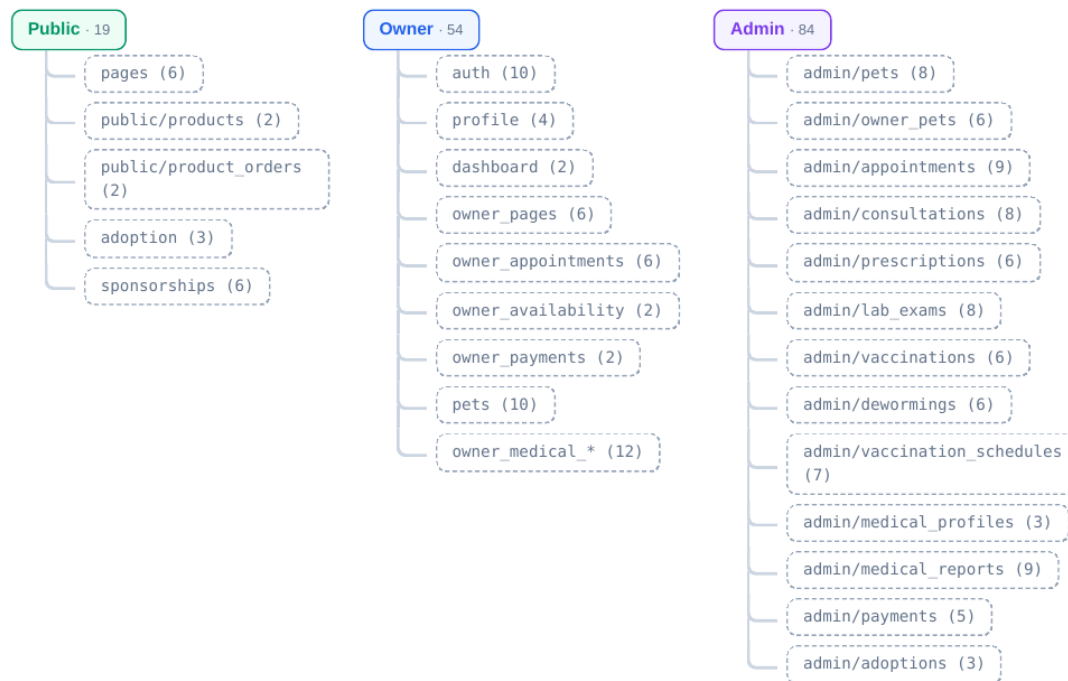
3. Diagrama de navegacion

RootNavigator decide el arbol segun el estado de sesion y el rol. Columna izquierda: acceso publico / pre-login. Columna derecha: navegadores autenticados (owner y staff).



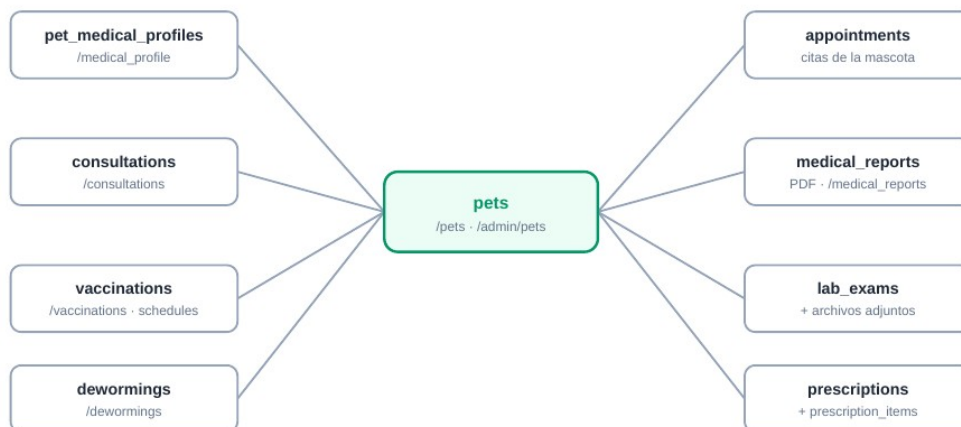
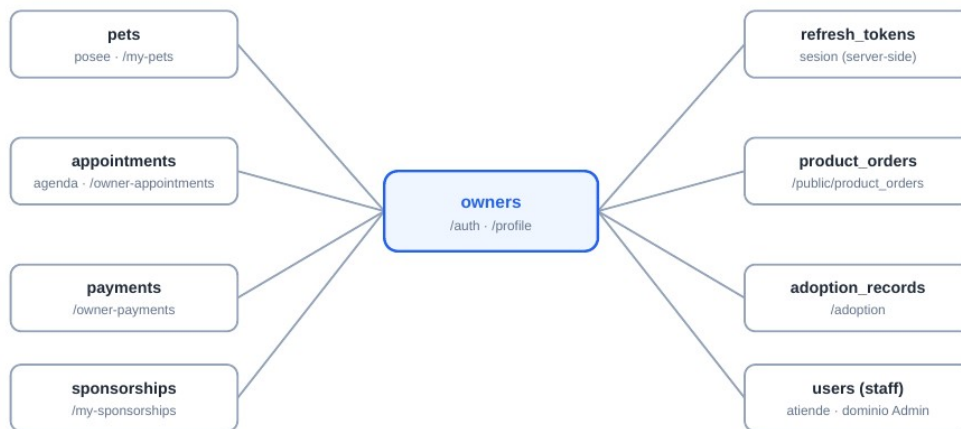
4. Diagrama de dominios y grupos de rutas

Las 157 rutas móviles agrupadas por dominio y controlador (numero de rutas entre parentesis). El grupo auth se consume a través de sus equivalentes dedicados /auth/mobile/* (Bearer, fase F4 de sesión móvil).



5. Diagrama de entidades (datos que toca la app movil)

Solo el subconjunto de tablas accesibles desde rutas moviles. Diagramas hub centrados en las dos entidades principales: Owner y Pet.



6. Diagrama de secuencia — sesion y seguridad

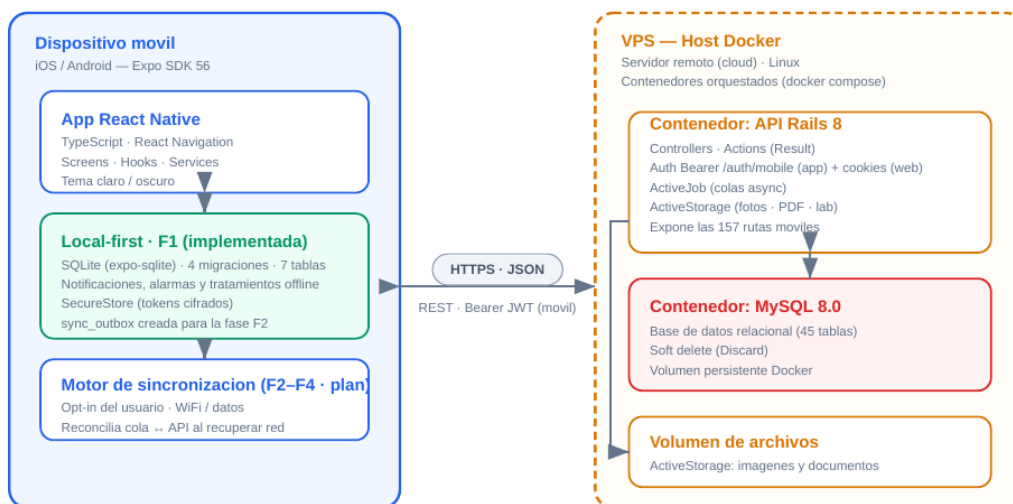
Manejo de tokens Bearer por /auth/mobile/* (RNF-SEC-001) y control de acceso por rol (RNF-SEC-002). Sesión móvil separada del flujo web por cookies.



Un dueño nunca accede a pantallas ni endpoints /admin/*; intentos no autorizados → 403 / redirección a login. Al cerrar sesión se purgan tokens y datos locales del usuario.

7. Diagrama de elementos del sistema (despliegue)

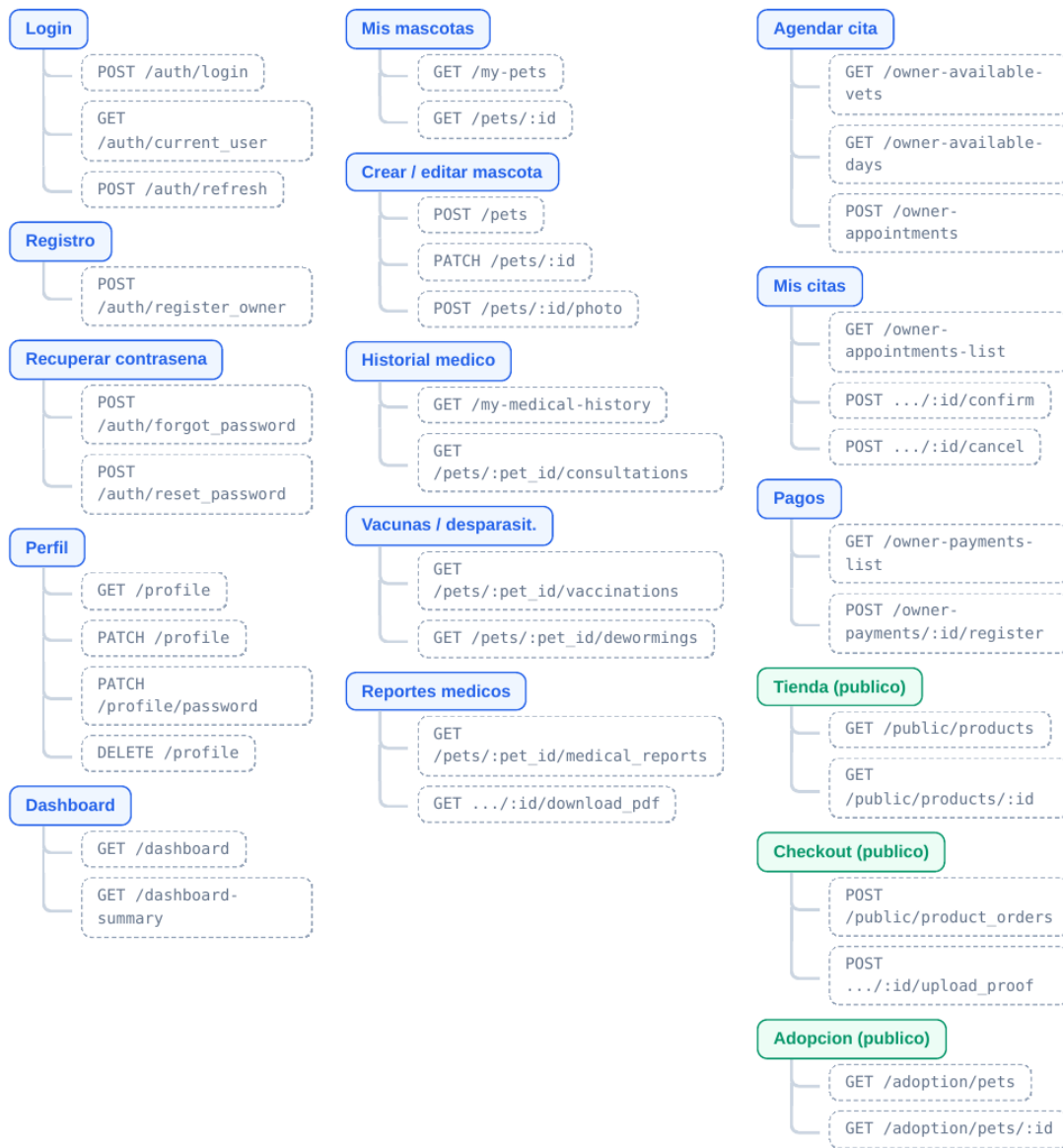
Nodos fisicos/logicos y su comunicacion. El dispositivo movil opera **local-first**; el backend Rails y MySQL corren en contenedores Docker.



El movil es **local-first**: la fase F1 (base SQLite verificada) ya opera notificaciones, alarmas y tratamientos sin conexion; la cola de escrituras y el motor de sincronizacion son las fases F2-F4 del ADR 0001 (continuidad). El backend Rails y MySQL se despliegan como contenedores Docker independientes sobre un **VPS**; la app permite ademas apuntar a un **servidor autoalojado** mediante el selector de servidor.

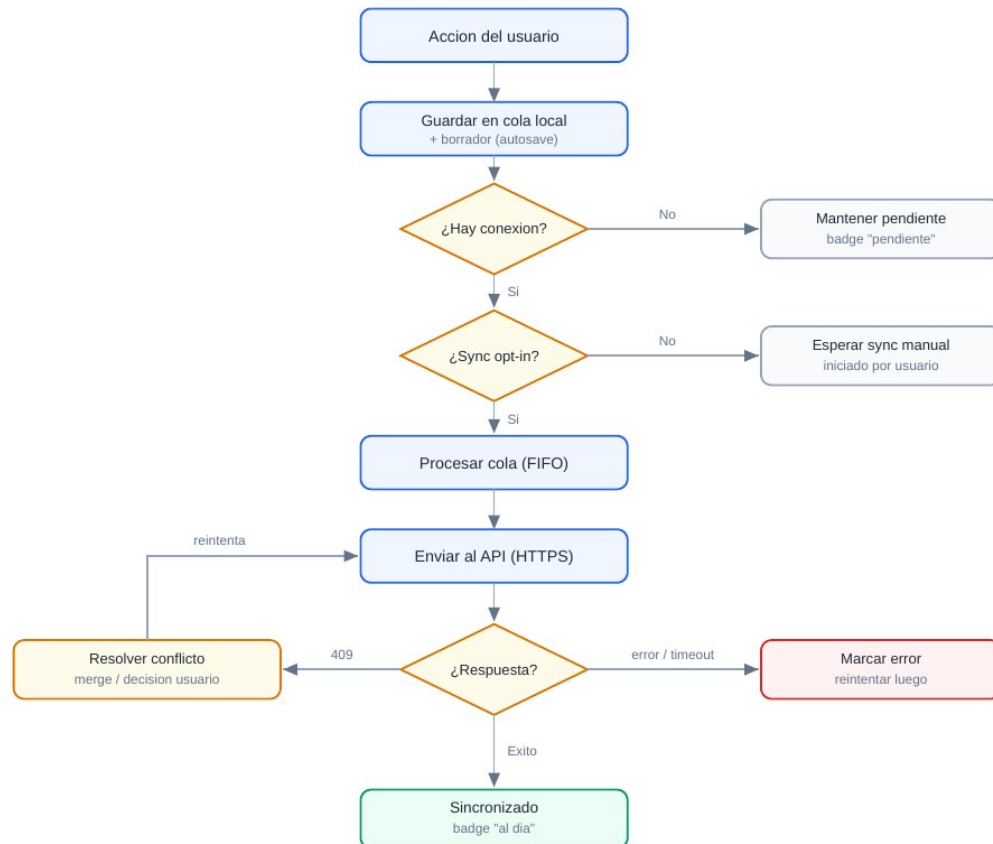
8. Mapa pantalla → endpoints

Cada pantalla principal con las rutas API que dispara. Verde = publico (sin auth) · azul = owner autenticado.



9. Diagrama de sincronizacion (local-first · disen0 F2–F4)

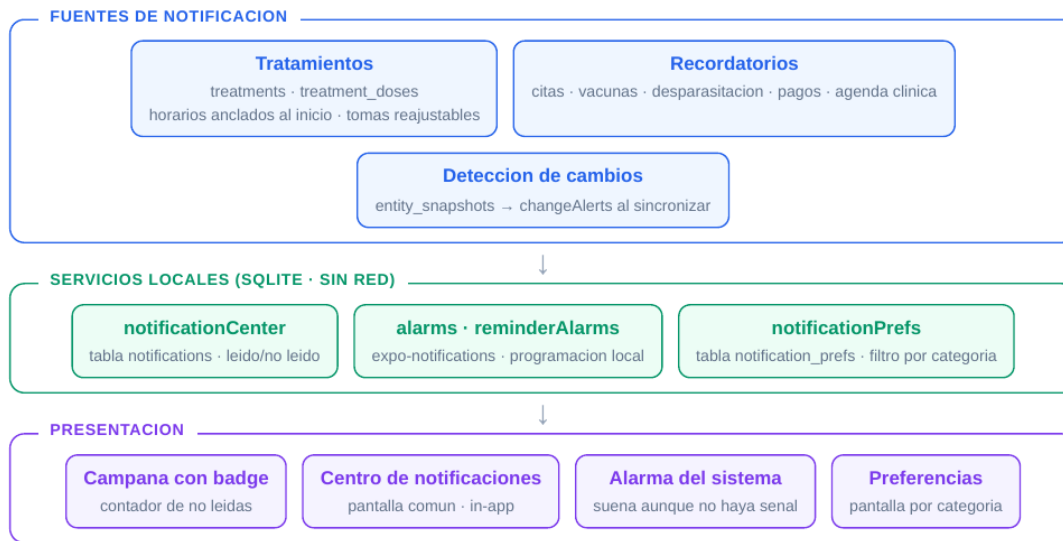
Flujo de la cola offline y el motor de sincronizacion (RNF-MOB-002, RNF-REL-002/003). Rombos = decision. **Estado:** disen0 del ADR 0001; la fase F1 (base SQLite + tabla sync_outbox) esta implementada y las fases F2–F4 quedan como continuidad del proyecto.



Sin conexion o sin opt-in, la app nunca simula exito: las escrituras quedan visibles como **pendientes** hasta confirmacion del servidor (RNF-REL-003).

10. Diagrama del sistema de notificaciones (local-first · sprint 4)

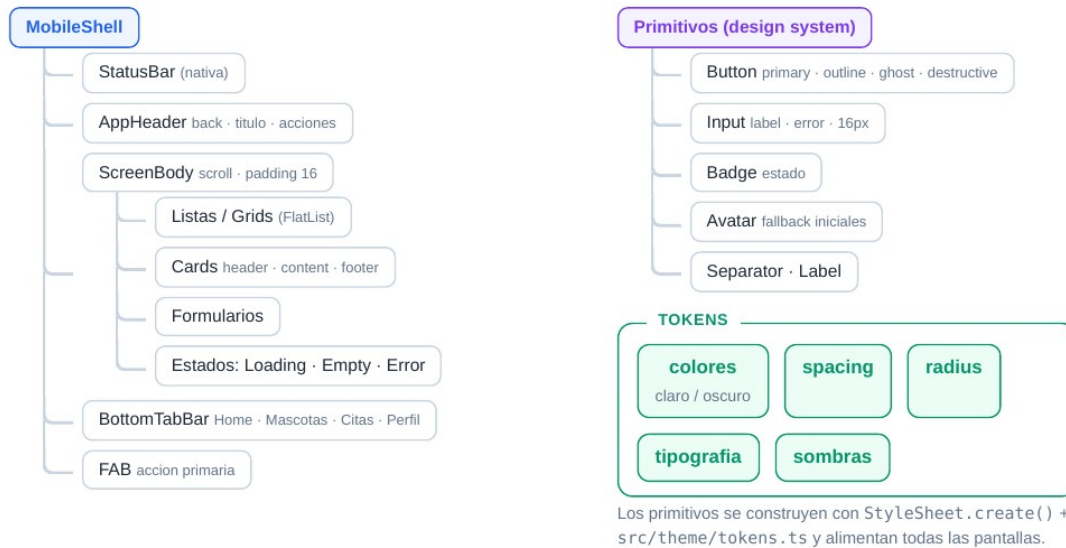
Todo el sistema opera **sin conexion** sobre la base SQLite local (fase F1). Las fuentes generan notificaciones que el centro in-app muestra y las preferencias por categoria filtran.



Las alarmas de medicacion son la prueba tangible del enfoque local-first: el tratamiento de la mascota no depende de la red. Las push remotas (app cerrada) quedan planificadas en el plan 033 del backend.

11. Diagrama de componentes UI

Jerarquía de composición (de `mobile-ui-components.md`). Los primitivos consumen los tokens del design system.



ANEXO D

Rutas del API Rails consumidas por la aplicación

Fuente: docs/rails-routes.txt — generado con `rails routes` desde el backend

Prefix	Verb	URI Pattern	Controller#Action
rails_health_check	GET	/up(.:format)	rails/health#show
health	GET	/health(.:format)	pages#health
root	GET	/	pages#home
unauthorized	GET	/unauthorized(.:format)	pages#unauthorized
not_found	GET	/not-found(.:format)	pages#not_found
landing	GET	/landing(.:format)	public/landing#index
public_contact_messages	POST	/public/ contact_messages(.:format)	public/ contact_messages#create
public_products	GET	/public/products(.:format)	public/products#index
public_product	GET	/public/products/:id(.:format)	public/products#show
upload_proof_public_product_order	POST	/public/product_orders/:id/ upload_proof(.:format)	public/ product_orders#upload_proof
public_product_orders	POST	/public/ product_orders(.:format)	public/product_orders#create
products	GET	/products(.:format)	pages#products
checkout	GET	/checkout(.:format)	pages#checkout
services	GET	/services(.:format)	pages#services
about	GET	/about(.:format)	pages#about
contact	GET	/contact(.:format)	pages#contact
terms	GET	/terms(.:format)	pages#terms
privacy	GET	/privacy(.:format)	pages#privacy
pawcare	GET	/pawcare(.:format)	pages#pawcare
adoption	GET	/adoption(.:format)	adoption#index
adoption_pets	GET	/adoption/pets(.:format)	adoption#pets
adoption_pet	GET	/adoption/pets/:id(.:format)	adoption#show
auth	GET	/auth(.:format)	auth#index
register_owner	POST	/auth/register_owner(.:format)	auth#register_owner
login	POST	/auth/login(.:format)	auth#login
logout	DELETE	/auth/logout(.:format)	auth#logout
refresh	POST	/auth/refresh(.:format)	auth#refresh
current_user	GET	/auth/current_user(.:format)	auth#current_user
debug	GET	/auth/debug(.:format)	auth#debug
forgot_password	POST	/auth/ forgot_password(.:format)	auth#forgot_password
reset_password	POST	/auth/reset_password(.:format)	auth#reset_password
edit_password_reset	GET	/auth/ reset_password/:token(.:format)	auth#edit_password_reset
profile	GET	/profile(.:format)	profile#show
	PATCH	/profile(.:format)	profile#update
profile_password	PATCH	/profile/password(.:format)	profile#change_password
	DELETE	/profile(.:format)	profile#destroy
dashboard	GET	/dashboard(.:format)	dashboard#index
dashboard_summary	GET	/dashboard-summary(.:format)	dashboard#summary
my_pets	GET	/my-pets(.:format)	owner_pages#pets
owner_medical_history	GET	/my-pets/:pet_id/medical- history(.:format)	owner_pages#medical_history
my_sponsorships	GET	/my-sponsorships(.:format)	owner_pages#sponsorships
my_appointments	GET	/my-appointments(.:format)	owner_pages#appointments
my_payments	GET	/my-payments(.:format)	owner_pages#payments
my_medical_history	GET	/my-medical-history(.:format)	owner_pages#medical_history_ index
owner_appointments_list	GET	/owner-appointments- list(.:format)	owner_appointments#index

owner_appointment	GET	/owner-appointments/:id(.:format)	owner_appointments#show
owner_appointments	POST	/owner-appointments(.:format)	owner_appointments#create
	PATCH	/owner-appointments/:id(.:format)	owner_appointments#update
owner_appointment_cancel	POST	/owner-appointments/:id/cancel(.:format)	owner_appointments#cancel
owner_appointment_confirm	POST	/owner-appointments/:id/confirm(.:format)	owner_appointments#confirm
owner_available_vets	GET	/owner-available-vets(.:format)	owner_availability#index
owner_available_days	GET	/owner-available-days(.:format)	owner_availability#days
owner_services_list	GET	/owner-services-list(.:format)	owner_services#index
owner_payments_list	GET	/owner-payments-list(.:format)	owner_payments#index
owner_payment	GET	/owner-payments/:id(.:format)	owner_payments#show
owner_payment_register	POST	/owner-payments/:id/register(.:format)	owner_payments#register
owner_medical_summary	GET	/owner-medical-summary(.:format)	owner_medical_summary#index
owner_consultation_export_recipe	GET	/owner/consultations/:id/export_recipe(.:format)	owner_consultations#export_recipe
owner_consultation_complete_treatment	POST	/owner/consultations/:id/complete_treatment(.:format)	owner_consultations#complete_treatment
owner_prescription_item_complete	POST	/owner/prescription_items/:id/complete(.:format)	owner_prescription_items#complete
owner_lab_exam_files	POST	/owner/lab_exams/:id/files(.:format)	owner_lab_exams#upload_files
owner_lab_exam_results	PATCH	/owner/lab_exams/:id/results(.:format)	owner_lab_exams#update_results
photo_pet	POST	/pets/:id/photo(.:format)	pets#photo
	DELETE	/pets/:id/photo(.:format)	pets#delete_photo
pet_medical_profile	GET	/pets/:pet_id/medical_profile(.:format)	owner_medical_profiles#show
pet_consultations	GET	/pets/:pet_id/consultations(.:format)	owner_consultations#index
pet_vaccinations	GET	/pets/:pet_id/vaccinations(.:format)	owner_vaccinations#index
pet_dewormings	GET	/pets/:pet_id/dewormings(.:format)	owner_dewormings#index
download_pdf_pet_medical_report	GET	/pets/:pet_id/medical_reports/:id/download_pdf(.:format)	owner_medical_reports#download_pdf
pet_medical_reports	GET	/pets/:pet_id/medical_reports(.:format)	owner_medical_reports#index
pet_medical_report	GET	/pets/:pet_id/medical_reports/:id(.:format)	owner_medical_reports#show
pets	GET	/pets(.:format)	pets#index
	POST	/pets(.:format)	pets#create
new_pet	GET	/pets/new(.:format)	pets#new
edit_pet	GET	/pets/:id/edit(.:format)	pets#edit
pet	GET	/pets/:id(.:format)	pets#show
	PATCH	/pets/:id(.:format)	pets#update
	PUT	/pets/:id(.:format)	pets#update
	DELETE	/pets/:id(.:format)	pets#destroy
expense_sponsorship	GET	/sponsorships/:id/expenses/:expense_id(.:format)	sponsorships#expense
sponsorships	GET	/sponsorships(.:format)	sponsorships#index
	POST	/sponsorships(.:format)	sponsorships#create
sponsorship	GET	/sponsorships/:id(.:format)	sponsorships#show
	PATCH	/sponsorships/:id(.:format)	sponsorships#update
	PUT	/sponsorships/:id(.:format)	sponsorships#update
admin_pets_page	GET	/admin/pets(.:format)	admin_pages#pets
admin_adoptions_page	GET	/admin/adoptions(.:format)	admin_pages#adoptions

admin_sponsorships_page	GET	/admin/sponsorships(:format)	admin_pages#sponsorships
admin_appointments_page	GET	/admin/appointments(:format)	admin_pages#appointments
admin_owners_page	GET	/admin/owners(:format)	admin_pages#owners
admin_medical_history_page	GET	/admin/medical-history(:format)	admin_pages#medical_records
admin_new_consultation_page	GET	/admin/new-consultation(:format)	admin_pages#consultations
admin_medical_reports_page	GET	/admin/medical-reports(:format)	admin_pages#reports
admin_payments_page	GET	/admin/payments(:format)	admin_pages#payments
admin_staff_page	GET	/admin/staff(:format)	admin_pages#staff
admin_metrics_page	GET	/admin/metrics(:format)	admin_pages#metrics
admin_shifts_page	GET	/admin/shifts(:format)	admin_pages#shifts
admin_financial_page	GET	/admin/financial(:format)	admin_pages#financial
admin_content_page	GET	/admin/content(:format)	admin_pages#content
admin_products_services_page	GET	/admin/products-services(:format)	admin_pages#products_service s
admin_email_settings_page	GET	/admin/settings/email(:format)	admin_pages#email_settings
admin_smtp_settings_page	GET	/admin/settings/smtp(:format)	admin_pages#smtp_settings
admin_currency_settings_page	GET	/admin/settings/currency(:format)	admin_pages#currency_setting s
admin_general_settings_page	GET	/admin/settings/general(:format)	admin_pages#general_settings
reorder_admin_content_promotions	PATCH	/admin/content/promotions/reorder(:format)	admin/content/promotions#reorder
admin_content_promotions	GET	/admin/content/promotions(:format)	admin/content/promotions#index
	POST	/admin/content/promotions(:format)	admin/content/promotions#create
new_admin_content_promotion	GET	/admin/content/promotions/new(:format)	admin/content/promotions#new
edit_admin_content_promotion	GET	/admin/content/promotions/:id/edit(:format)	admin/content/promotions#edit
admin_content_promotion	GET	/admin/content/promotions/:id(:format)	admin/content/promotions#show
	PATCH	/admin/content/promotions/:id(:format)	admin/content/promotions#update
	PUT	/admin/content/promotions/:id(:format)	admin/content/promotions#update
	DELETE	/admin/content/promotions/:id(:format)	admin/content/promotions#destroy
admin_content_announcements	GET	/admin/content/announcements(:format)	admin/content/announcements#index
	POST	/admin/content/announcements(:format)	admin/content/announcements#create
new_admin_content_announcement	GET	/admin/content/announcements/new(:format)	admin/content/announcements#new
edit_admin_content_announcement	GET	/admin/content/announcements/:id/edit(:format)	admin/content/announcements#edit
admin_content_announcement	GET	/admin/content/announcements/:id(:format)	admin/content/announcements#show
	PATCH	/admin/content/announcements/:id(:format)	admin/content/announcements#update
	PUT	/admin/content/announcements/:id(:format)	admin/content/announcements#update
	DELETE	/admin/content/announcements/:id(:format)	admin/content/announcements#destroy
admin_content_landing_config	GET	/admin/content/landing_config(:format)	admin/content/landing_configs#show
	PATCH	/admin/content/landing_config(:format)	admin/content/landing_configs#update
	PUT	/admin/content/landing_config(:format)	admin/content/landing_configs#update
preview_recipients_admin_con	POST	/admin/content/	admin/content/

tent_email_campaigns		email_campaigns/ preview_recipients(.:format)	email_campaigns#preview_recipients
send_campaign_admin_content_email_campaign	POST	/admin/content/ email_campaigns/:id/ send_campaign(.:format)	admin/content/ email_campaigns#send_campaign
cancel_admin_content_email_campaign	POST	/admin/content/ email_campaigns/:id/ cancel(.:format)	admin/content/ email_campaigns#cancel
clone_admin_content_email_campaign	POST	/admin/content/ email_campaigns/:id/ clone(.:format)	admin/content/ email_campaigns#clone
admin_content_email_campaigns	GET	/admin/content/ email_campaigns(.:format)	admin/content/ email_campaigns#index
	POST	/admin/content/ email_campaigns(.:format)	admin/content/ email_campaigns#create
new_admin_content_email_campaign	GET	/admin/content/ email_campaigns/new(.:format)	admin/content/ email_campaigns#new
edit_admin_content_email_campaign	GET	/admin/content/ email_campaigns/:id/ edit(.:format)	admin/content/ email_campaigns#edit
admin_content_email_campaign	GET	/admin/content/ email_campaigns/:id(.:format)	admin/content/ email_campaigns#show
	PATCH	/admin/content/ email_campaigns/:id(.:format)	admin/content/ email_campaigns#update
	PUT	/admin/content/ email_campaigns/:id(.:format)	admin/content/ email_campaigns#update
	DELETE	/admin/content/ email_campaigns/:id(.:format)	admin/content/ email_campaigns#destroy
reorder_admin_content_testimonials	PATCH	/admin/content/testimonials/ reorder(.:format)	admin/content/ testimonials#reorder
admin_content_testimonials	GET	/admin/content/ testimonials(.:format)	admin/content/ testimonials#index
	POST	/admin/content/ testimonials(.:format)	admin/content/ testimonials#create
new_admin_content_testimonial	GET	/admin/content/testimonials/ new(.:format)	admin/content/ testimonials#new
edit_admin_content_testimonial	GET	/admin/content/ testimonials/:id/edit(.:format)	admin/content/ testimonials#edit
admin_content_testimonial	GET	/admin/content/ testimonials/:id(.:format)	admin/content/ testimonials#show
	PATCH	/admin/content/ testimonials/:id(.:format)	admin/content/ testimonials#update
	PUT	/admin/content/ testimonials/:id(.:format)	admin/content/ testimonials#update
	DELETE	/admin/content/ testimonials/:id(.:format)	admin/content/ testimonials#destroy
admin_pets_list	GET	/admin/pets-list(.:format)	admin/pets#index
admin_adoptions_list	GET	/admin/adoptions-list(.:format)	admin/adoptions#index
admin_owners_list	GET	/admin/owners-list(.:format)	admin/owners#index
admin_sponsorships_list	GET	/admin/sponsorships- list(.:format)	admin/sponsorships#index
admin_consultations_list	GET	/admin/consultations- list(.:format)	admin/consultations#index
admin_vaccinations_list	GET	/admin/vaccinations- list(.:format)	admin/vaccinations#index
admin_dewormings_list	GET	/admin/dewormings- list(.:format)	admin/dewormings#index
admin_vaccination_schedules_list	GET	/admin/vaccination-schedules- list(.:format)	admin/ vaccination_schedules#index
admin_staff_list	GET	/admin/staff-list(.:format)	admin/staff#index
admin_services_list	GET	/admin/services-list(.:format)	admin/services#index
admin_staff_schedules_list	GET	/admin/staff-schedules- list(.:format)	admin/staff_schedules#index
admin_appointments_list	GET	/admin/appointments- list(.:format)	admin/appointments#index
admin_appointments_available	GET	/admin/appointments/	admin/

_days		available-days(:format)	appointments#available_days
admin_appointments_available_vets	GET	/admin/appointments/available-vets(:format)	admin/appointments#available_vets
admin_appointments_time_slots	GET	/admin/appointments/time-slots(:format)	admin/appointments#time_slots
admin_payments_list	GET	/admin/payments-list(:format)	admin/payments#index
admin_products_list	GET	/admin/products-list(:format)	admin/products#index
admin_stock_movements_list	GET	/admin/stock-movements-list(:format)	admin/stock_movements#index
admin_service_categories_list	GET	/admin/service-categories-list(:format)	admin/service_categories#index
admin_email_settings_config	GET	/admin/settings/email-config(:format)	admin/settings#email
admin_settings_email_config	PATCH	/admin/settings/email-config(:format)	admin/settings#update_email
admin_smtp_settings_config	GET	/admin/settings/smtp-config(:format)	admin/settings#smtp
admin_settings_smtp_config	PATCH	/admin/settings/smtp-config(:format)	admin/settings#update_smtp
admin_smtp_settings_test	POST	/admin/settings/smtp-config/test(:format)	admin/settings#test_smtp
admin_currency_settings_config	GET	/admin/settings/currency-config(:format)	admin/settings#currency
admin_settings_currency_config	PATCH	/admin/settings/currency-config(:format)	admin/settings#update_currency
admin_general_settings_config	GET	/admin/settings/general-config(:format)	admin/settings#general
admin_settings_general_config	PATCH	/admin/settings/general-config(:format)	admin/settings#update_general
admin_smtp_status	GET	/admin/settings/smtp-status(:format)	admin/settings#smtp_status
photo_admin_owner_pet	POST	/admin/owners/:owner_id/pets/:id/photo(:format)	admin/owner_pets#photo
	DELETE	/admin/owners/:owner_id/pets/:id/photo(:format)	admin/owner_pets#delete_photo
admin_owner_pets	POST	/admin/owners/:owner_id/pets(:format)	admin/owner_pets#create
admin_owner_pet	GET	/admin/owners/:owner_id/pets/:id(:format)	admin/owner_pets#show
	PATCH	/admin/owners/:owner_id/pets/:id(:format)	admin/owner_pets#update
	PUT	/admin/owners/:owner_id/pets/:id(:format)	admin/owner_pets#update
admin_owners	POST	/admin/owners(:format)	admin/owners#create
admin_owner	GET	/admin/owners/:id(:format)	admin/owners#show
	PATCH	/admin/owners/:id(:format)	admin/owners#update
	PUT	/admin/owners/:id(:format)	admin/owners#update
admin_staff_index	POST	/admin/staff(:format)	admin/staff#create
admin_staff	GET	/admin/staff/:id(:format)	admin/staff#show
	PATCH	/admin/staff/:id(:format)	admin/staff#update
	PUT	/admin/staff/:id(:format)	admin/staff#update
	DELETE	/admin/staff/:id(:format)	admin/staff#destroy
admin_services	POST	/admin/services(:format)	admin/services#create
admin_service	GET	/admin/services/:id(:format)	admin/services#show
	PATCH	/admin/services/:id(:format)	admin/services#update
	PUT	/admin/services/:id(:format)	admin/services#update
	DELETE	/admin/services/:id(:format)	admin/services#destroy
admin_service_categories	POST	/admin/service_categories(:format)	admin/service_categories#create
admin_service_category	GET	/admin/service_categories/:id(:format)	admin/service_categories#show

	PATCH	/admin/ service_categories/:id(.:format)	admin/ service_categories#update
	PUT	/admin/ service_categories/:id(.:format)	admin/ service_categories#update
	DELETE	/admin/ service_categories/:id(.:format)	admin/ service_categories#destroy
add_stock_admin_product	POST	/admin/products/:id/ add_stock(.:format)	admin/products#add_stock
remove_stock_admin_product	POST	/admin/products/:id/ remove_stock(.:format)	admin/products#remove_stock
adjust_stock_admin_product	POST	/admin/products/:id/ adjust_stock(.:format)	admin/products#adjust_stock
image_admin_product	POST	/admin/products/:id/ image(.:format)	admin/products#image
	DELETE	/admin/products/:id/ image(.:format)	admin/products#delete_image
admin_products	POST	/admin/products(.:format)	admin/products#create
admin_product	GET	/admin/products/:id(.:format)	admin/products#show
	PATCH	/admin/products/:id(.:format)	admin/products#update
	PUT	/admin/products/:id(.:format)	admin/products#update
	DELETE	/admin/products/:id(.:format)	admin/products#destroy
admin_stock_movement	GET	/admin/ stock_movements/:id(.:format)	admin/stock_movements#show
admin_staff_schedules	POST	/admin/ staff_schedules(.:format)	admin/staff_schedules#create
admin_staff_schedule	GET	/admin/ staff_schedules/:id(.:format)	admin/staff_schedules#show
	PATCH	/admin/ staff_schedules/:id(.:format)	admin/staff_schedules#update
	PUT	/admin/ staff_schedules/:id(.:format)	admin/staff_schedules#update
	DELETE	/admin/ staff_schedules/:id(.:format)	admin/staff_schedules#destroy
admin_appointments	POST	/admin/appointments(.:format)	admin/appointments#create
admin_appointment	GET	/admin/ appointments/:id(.:format)	admin/appointments#show
	PATCH	/admin/ appointments/:id(.:format)	admin/appointments#update
	PUT	/admin/ appointments/:id(.:format)	admin/appointments#update
	DELETE	/admin/ appointments/:id(.:format)	admin/appointments#destroy
register_admin_payment	POST	/admin/payments/:id/ register(.:format)	admin/payments#register
extend_admin_payment	POST	/admin/payments/:id/ extend(.:format)	admin/payments#extend
cancel_admin_payment	POST	/admin/payments/:id/ cancel(.:format)	admin/payments#cancel
admin_payments	POST	/admin/payments(.:format)	admin/payments#create
admin_payment	GET	/admin/payments/:id(.:format)	admin/payments#show
	PATCH	/admin/payments/:id(.:format)	admin/payments#update
	PUT	/admin/payments/:id(.:format)	admin/payments#update
photo_admin_pet	POST	/admin/pets/:id/photo(.:format)	admin/pets#photo
	DELETE	/admin/pets/:id/photo(.:format)	admin/pets#delete_photo
admin_pet_medical_profile	GET	/admin/pets/:pet_id/ medical_profile(.:format)	admin/medical_profiles#show
	PATCH	/admin/pets/:pet_id/ medical_profile(.:format)	admin/medical_profiles#update
	PUT	/admin/pets/:pet_id/ medical_profile(.:format)	admin/medical_profiles#update
admin_pets	POST	/admin/pets(.:format)	admin/pets#create
admin_pet	GET	/admin/pets/:id(.:format)	admin/pets#show

	PATCH	/admin/pets/:id(.:format)	admin/pets#update
	PUT	/admin/pets/:id(.:format)	admin/pets#update
	DELETE	/admin/pets/:id(.:format)	admin/pets#destroy
admin_adoptions	POST	/admin/adoptions(.:format)	admin/adoptions#create
admin_adoption	GET	/admin/adoptions/:id(.:format)	admin/adoptions#show
admin_sponsorship	GET	/admin/ sponsorships/:id(.:format)	admin/sponsorships#show
admin_complete_prescription_item	POST	/admin/prescription_items/:id/ complete(.:format)	admin/ prescription_items#complete
export_recipe_admin_consultation	GET	/admin/consultations/:id/ export_recipe(.:format)	admin/ consultations#export_recipe
complete_treatment_admin_consultation	POST	/admin/consultations/:id/ complete_treatment(.:format)	admin/ consultations#complete_treatment
admin_consultation_prescriptions	GET	/admin/ consultations/:consultation_id/ prescriptions(.:format)	admin/prescriptions#index
	POST	/admin/ consultations/:consultation_id/ prescriptions(.:format)	admin/prescriptions#create
admin_consultation_prescription	GET	/admin/ consultations/:consultation_id/ prescriptions/:id(.:format)	admin/prescriptions#show
	PATCH	/admin/ consultations/:consultation_id/ prescriptions/:id(.:format)	admin/prescriptions#update
	PUT	/admin/ consultations/:consultation_id/ prescriptions/:id(.:format)	admin/prescriptions#update
	DELETE	/admin/ consultations/:consultation_id/ prescriptions/:id(.:format)	admin/prescriptions#destroy
files_admin_consultation_lab_exam	POST	/admin/ consultations/:consultation_id/ lab_exams/:id/files(.:format)	admin/lab_exams#upload_files
	DELETE	/admin/ consultations/:consultation_id/ lab_exams/:id/ files/:file_id(.:format)	admin/lab_exams#delete_file
admin_consultation_lab_exams	GET	/admin/ consultations/:consultation_id/ lab_exams(.:format)	admin/lab_exams#index
	POST	/admin/ consultations/:consultation_id/ lab_exams(.:format)	admin/lab_exams#create
admin_consultation_lab_exam	GET	/admin/ consultations/:consultation_id/ lab_exams/:id(.:format)	admin/lab_exams#show
	PATCH	/admin/ consultations/:consultation_id/ lab_exams/:id(.:format)	admin/lab_exams#update
	PUT	/admin/ consultations/:consultation_id/ lab_exams/:id(.:format)	admin/lab_exams#update
	DELETE	/admin/ consultations/:consultation_id/ lab_exams/:id(.:format)	admin/lab_exams#destroy
admin_consultations	POST	/admin/consultations(.:format)	admin/consultations#create
admin_consultation	GET	/admin/ consultations/:id(.:format)	admin/consultations#show
	PATCH	/admin/ consultations/:id(.:format)	admin/consultations#update
	PUT	/admin/ consultations/:id(.:format)	admin/consultations#update

	DELETE	/admin/ consultations/:id(.:format)	admin/consultations#destroy
admin_vaccinations	POST	/admin/vaccinations(.:format)	admin/vaccinations#create
admin_vaccination	GET	/admin/ vaccinations/:id(.:format)	admin/vaccinations#show
	PATCH	/admin/ vaccinations/:id(.:format)	admin/vaccinations#update
	PUT	/admin/ vaccinations/:id(.:format)	admin/vaccinations#update
	DELETE	/admin/ vaccinations/:id(.:format)	admin/vaccinations#destroy
admin_dewormings	POST	/admin/dewormings(.:format)	admin/dewormings#create
admin_deworming	GET	/admin/ dewormings/:id(.:format)	admin/dewormings#show
	PATCH	/admin/ dewormings/:id(.:format)	admin/dewormings#update
	PUT	/admin/ dewormings/:id(.:format)	admin/dewormings#update
	DELETE	/admin/ dewormings/:id(.:format)	admin/dewormings#destroy
complete_item_admin_vaccinat ion_schedule	POST	/admin/ vaccination_schedules/:id/ complete_item(.:format)	admin/ vaccination_schedules#complet e_item
admin_vaccination_schedules	POST	/admin/ vaccination_schedules(.:format)	admin/ vaccination_schedules#create
admin_vaccination_schedule	GET	/admin/ vaccination_schedules/:id(.:for mat)	admin/ vaccination_schedules#show
	PATCH	/admin/ vaccination_schedules/:id(.:for mat)	admin/ vaccination_schedules#update
	PUT	/admin/ vaccination_schedules/:id(.:for mat)	admin/ vaccination_schedules#update
	DELETE	/admin/ vaccination_schedules/:id(.:for mat)	admin/ vaccination_schedules#destroy
generate_pdf_admin_medical_r eports	GET	/admin/medical_reports/ generate_pdf(.:format)	admin/ medical_reports#generate_pdf
import_admin_medical_reports	POST	/admin/medical_reports/ import(.:format)	admin/medical_reports#import
export_json_admin_medical_re port	GET	/admin/medical_reports/:id/ export_json(.:format)	admin/ medical_reports#export_json
export_csv_admin_medical_rep ort	GET	/admin/medical_reports/:id/ export_csv(.:format)	admin/ medical_reports#export_csv
export_pdf_admin_medical_rep ort	GET	/admin/medical_reports/:id/ export_pdf(.:format)	admin/ medical_reports#export_pdf
email_admin_medical_report	POST	/admin/medical_reports/:id/ email(.:format)	admin/medical_reports#email
admin_medical_reports	GET	/admin/ medical_reports(.:format)	admin/medical_reports#index
	POST	/admin/ medical_reports(.:format)	admin/medical_reports#create
admin_medical_report	GET	/admin/ medical_reports/:id(.:format)	admin/medical_reports#show
admin_metrics_dashboard	GET	/admin/metrics/ dashboard(.:format)	admin/metrics#dashboard
admin_metrics_consultations	GET	/admin/metrics/ consultations(.:format)	admin/metrics#consultations
admin_metrics_patients	GET	/admin/metrics/ patients(.:format)	admin/metrics#patients
admin_metrics_revenue	GET	/admin/metrics/ revenue(.:format)	admin/metrics#revenue
admin_metrics_appointments	GET	/admin/metrics/ appointments(.:format)	admin/metrics#appointments
admin_metrics_vaccinations	GET	/admin/metrics/	admin/metrics#vaccinations

		vaccinations(.:format)	
admin_metrics_adoptions	GET	/admin/metrics/ adoptions(.:format)	admin/metrics#adoptions
admin_metrics_sponsorships	GET	/admin/metrics/ sponsorships(.:format)	admin/metrics#sponsorships
admin_metrics_financial	GET	/admin/metrics/ financial(.:format)	admin/metrics#financial
admin_metrics_refresh	POST	/admin/metrics/ refresh(.:format)	admin/metrics#refresh
admin_shifts_clinic_schedule	GET	/admin/shifts/ clinic_schedule(.:format)	admin/shifts#clinic_schedule
	PATCH	/admin/shifts/ clinic_schedule(.:format)	admin/ shifts#update_clinic_schedule
admin_shifts_staff_schedules	GET	/admin/shifts/ staff_schedules(.:format)	admin/shifts#staff_schedules
admin_shifts	PATCH	/admin/shifts/ staff_schedules/:user_id(.:format)	admin/ shifts#update_staff_schedule
admin_shifts_weekly_overview	GET	/admin/shifts/ weekly_overview(.:format)	admin/shifts#weekly_overview
admin_shifts_exceptions	POST	/admin/shifts/ exceptions(.:format)	admin/shifts#create_exception
admin_shifts_exception	DELETE	/admin/shifts/ exceptions/:id(.:format)	admin/ shifts#destroy_exception
admin_financial_expenses	GET	/admin/financial/ expenses(.:format)	admin/financial#index
admin_financial_expense	GET	/admin/financial/ expenses/:id(.:format)	admin/financial#show
admin_expenses	POST	/admin/financial/ expenses(.:format)	admin/financial#create
admin	PATCH	/admin/financial/ expenses/:id(.:format)	admin/financial#update
	DELETE	/admin/financial/ expenses/:id(.:format)	admin/financial#destroy
admin_financial_balance	GET	/admin/financial/ balance(.:format)	admin/financial#balance
admin_financial_refunds	GET	/admin/financial/ refunds(.:format)	admin/financial#refunds_index
admin_refunds	POST	/admin/financial/ refunds(.:format)	admin/ financial#refunds_create
	POST	/auth/mobile/login(.:format)	auth/mobile#login
register	POST	/auth/mobile/register(.:format)	auth/mobile#register
	POST	/auth/mobile/refresh(.:format)	auth/mobile#refresh
me	GET	/auth/mobile/me(.:format)	auth/mobile#me
	DELETE	/auth/mobile/logout(.:format)	auth/mobile#logout
	POST	/auth/mobile/ forgot_password(.:format)	auth/mobile#forgot_password
	POST	/auth/mobile/ reset_password(.:format)	auth/mobile#reset_password
	GET	/auth/mobile/ reset_password/:token(.:format)	auth/ mobile#validate_reset_token
devices	POST	/auth/mobile/devices(.:format)	auth/mobile#register_device
	PATCH	/auth/mobile/devices(.:format)	auth/mobile#update_device
	DELETE	/auth/mobile/devices(.:format)	auth/mobile#unregister_device
		/*path(.:format)	pages#not_found
rails_postmark_inbound_emails	POST	/rails/action_mailbox/ postmark/ inbound_emails(.:format)	action_mailbox/ingresses/ postmark/ inbound_emails#create
rails_relay_inbound_emails	POST	/rails/action_mailbox/relay/ inbound_emails(.:format)	action_mailbox/ingresses/ relay/inbound_emails#create
rails_sendgrid_inbound_emails	POST	/rails/action_mailbox/sendgrid/ inbound_emails(.:format)	action_mailbox/ingresses/ sendgrid/

			inbound_emails#create
rails_mandrill_inbound_health_check	GET	/rails/action_mailbox/mandrill/inbound_emails(.:format)	action_mailbox/ingresses/mandrill/inbound_emails#health_check
rails_mandrill_inbound_emails	POST	/rails/action_mailbox/mandrill/inbound_emails(.:format)	action_mailbox/ingresses/mandrill/inbound_emails#create
rails_mailgun_inbound_emails	POST	/rails/action_mailbox/mailgun/inbound_emails/mime(.:format)	action_mailbox/ingresses/mailgun/inbound_emails#create
rails_conductor_inbound_emails	GET	/rails/conductor/action_mailbox/inbound_emails(.:format)	rails/conductor/action_mailbox/inbound_emails#index
	POST	/rails/conductor/action_mailbox/inbound_emails(.:format)	rails/conductor/action_mailbox/inbound_emails#create
new_rails_conductor_inbound_email	GET	/rails/conductor/action_mailbox/inbound_emails/new(.:format)	rails/conductor/action_mailbox/inbound_emails#new
rails_conductor_inbound_email	GET	/rails/conductor/action_mailbox/inbound_emails/:id(.:format)	rails/conductor/action_mailbox/inbound_emails#show
new_rails_conductor_inbound_email_source	GET	/rails/conductor/action_mailbox/inbound_emails/sources/new(.:format)	rails/conductor/action_mailbox/inbound_emails/sources#new
rails_conductor_inbound_email_sources	POST	/rails/conductor/action_mailbox/inbound_emails/sources(.:format)	rails/conductor/action_mailbox/inbound_emails/sources#create
rails_conductor_inbound_email_reroute	POST	/rails/conductor/action_mailbox/inbound_email_id/reroute(.:format)	rails/conductor/action_mailbox/reroutes#create
rails_conductor_inbound_email_incinerate	POST	/rails/conductor/action_mailbox/inbound_email_id/incinerate(.:format)	rails/conductor/action_mailbox/incinerates#create
rails_service_blob	GET	/rails/active_storage/blobs/redirect/:signed_id/*filename(.:format)	active_storage/blobs/redirect#show
rails_service_blob_proxy	GET	/rails/active_storage/blobs/proxy/:signed_id/*filename(.:format)	active_storage/blobs/proxy#show
	GET	/rails/active_storage/blobs/:signed_id/*filename(.:format)	active_storage/blobs/redirect#show
rails_blob_representation	GET	/rails/active_storage/representations/redirect/:signed_blob_id/:variation_key/*filename(.:format)	active_storage/representations/redirect#show
rails_blob_representation_proxy	GET	/rails/active_storage/representations/proxy/:signed_blob_id/:variation_key/*filename(.:format)	active_storage/representations/proxy#show
	GET	/rails/active_storage/representations/:signed_blob_id/:variation_key/*filename(.:format)	active_storage/representations/redirect#show
rails_disk_service	GET	/rails/active_storage/disk/:encoded_key/*filename(.:format)	active_storage/disk#show
update_rails_disk_service	PUT	/rails/active_storage/disk/:encoded_token(.:format)	active_storage/disk#update
rails_direct_uploads	POST	/rails/active_storage/direct_uploads(.:format)	active_storage/direct_uploads#create